
Python multibody4everybody

Release v1

Alvaro Diaz Flores Caminero, Sahand Sabet

Jul 30, 2025

CORE MODULES

1	Executive Summary	2
1.1	Why a New Multibody Tool?	3
1.2	How Multibody4Everybody Works	4
1.3	Validation and Help for New Users	5
1.4	Symbolic or Numeric—Your Choice	5
2	Installation	6
2.1	Documentation	7
3	User documentation	8
3.1	How to Write an Example	8
3.2	Quick examples overview:	9
3.3	Detailed examples	24
3.4	Validation of EOM derivation	49
4	Developer’s documentation	56
4.1	Project Layout	56
4.2	Public API summary	57
4.3	Multibody system creation workflow	58
4.4	PyTests	59
4.5	Public API	60
	Python Module Index	89
	Index	90

EXECUTIVE SUMMARY

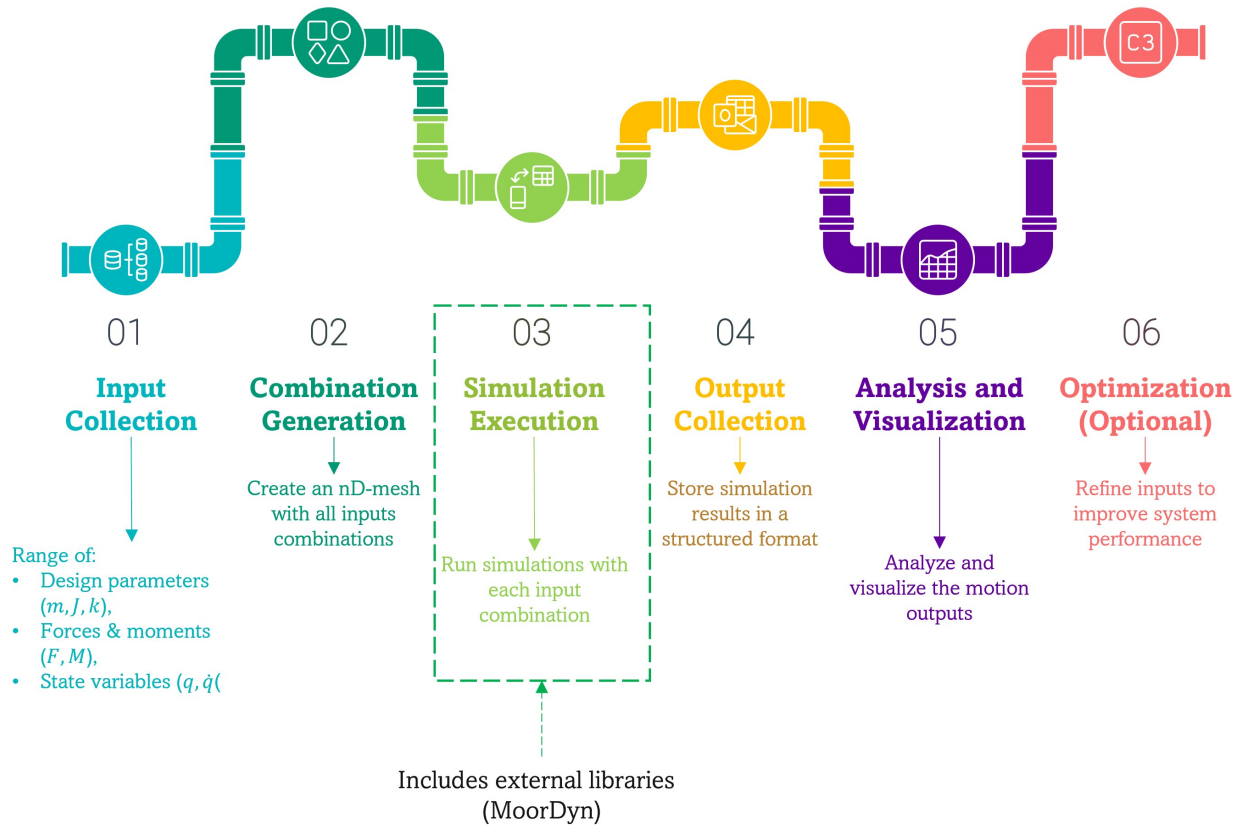
Welcome to Multibody4Everybody, a symbolic multibody dynamics engine designed to be accessible, efficient, and extensible. Multibody dynamics (MBD) allows us to model mechanical systems composed of interconnected rigid (or flexible) bodies.

These systems are common in robotics, biomechanics, aerospace, marine engineering, and many other fields. However, many existing tools for MBD are either closed-source, difficult to extend, or require advanced knowledge to use effectively.

This project was developed to provide a user-friendly and symbolic multibody framework that simplifies the modeling process while retaining full flexibility and mathematical transparency. Additionally, this code is an efficient tool for design exploration and optimization, evaluating constraint forces at the joints based on all combinations of degrees of freedom.

At a glance, this is what is possible with Multibody4Everybody design optimization:

Multibody Framework Simulation Process



Note: CLOSED CHAIN CAPABILITIES NOT YET ADDED

1.1 Why a New Multibody Tool?

Several open-source MBD libraries already exist:

- Chrono is a powerful C++-based library with many add-on modules, but it does not provide symbolic equations of motion or use reduced-coordinate formulations.
- PyDy (Python Dynamics) uses Kane's method and offers symbolic equation generation—but it often requires deep familiarity with multibody theory and SymPy mechanics.

In contrast, Multibody4Everybody is built for:

- Ease of use, even for those new to MBD
- Symbolic generation of the equations of motion (EOMs) for design and optimization

- Efficient simulation using minimal (joint) coordinates
- Plug-and-play interfaces for external forces, including libraries like MoorDyn

1.2 How Multibody4Everybody Works

To use the code, all you need is a single input file. In short, you define the kinematics of the system (a snapshot at its initial condition) and the forces and points where forces are applied.

This file contains structured information about your mechanical system:

1. Define the reference frame and bodies

- Specify the location of the global origin (the y-axis is perpendicular to the working plane).
- Define the bodies and how they are connected (parent-child relationships).
- Define the center of mass (CG) of each body and the location of each joint relative to the CG (in the body-fixed frame).
- Describe the type of joint connecting each body (e.g., revolute, prismatic, floating).

2. Body Points (Optional)

- Add body points, defined in the body-fixed frame and relative to the CG of the corresponding body, which move with the body.
- Add ground points, in the global frame, which remain fixed. These can later be used for applying forces or constraints.

3. External Forces (Optional)

- Apply forces or torques to the CG or to body points defined above.
- Add springs, dampers, or mooring lines using a consistent format. The springs and dampers can be linear or nonlinear.
- Mooring systems can be defined via MoorDyn, using the same interface style as springs and dampers.

4. Numerical Parameters

- Provide initial conditions (positions and velocities).

- Set body properties like mass and inertia.
- Define simulation parameters: gravity, total simulation time, and time step.

The numerical parameters step is needed to simulate the system over time. The parameters defined above could have been symbolic or numeric—once everything is defined, the code can generate and simulate the system, animate its motion, and optionally save it as a GIF.

1.3 Validation and Help for New Users

Each example included in the package contains the structure described above, along with:

- Automatic checks to validate that joints, points, and forces are defined consistently.
- Table displays for joints, points, and forces—allowing visual inspection of each section.
- Initial condition guidance, showing which variables require starting values, even if you're unfamiliar with the joint coordinate method.

For clarity, each example ends with a short reference section explaining the expected format for each field, including valid values and data types. This documentation is designed to help both new and advanced users define systems correctly.

1.4 Symbolic or Numeric—Your Choice

A key strength of this package is flexibility: any quantity you define can be symbolic (for symbolic derivations) or numeric (for direct simulation). You can start simple and build complexity as needed.

This introduction is intended to help you get started quickly and confidently. If you're new to multibody modeling, don't worry—the examples and built-in checks will guide you.

INSTALLATION

The goal of this chapter is to help the user install the python package as an external library that can be used no matter where it is installed. This is common practice in the open-source community. To proceed, the user must open a terminal at the location where they want to download the package for installation, typically the downloads folder.

To install the package without prior knowledge we recommend following option number one: CONDA option. This requires having conda installed in the computer. For how to install conda please visit: [conda installation](#) Once conda is installed proceed with the following steps.

Common step: to install using either conda or pip, first run on the terminal:

```
# Clone the repo and enter the directory  
git clone https://github.com/Project-SEA-Stack/Python_multibody_dyamics.git  
cd Python_multibody_dyamics
```

Option 1: then for CONDA environment creation and installation run:

```
# Create and activate the Conda environment  
conda env create -f multibody_env.yaml  
conda activate multibody_env
```

Option 2: for PIP installation in an already existing environment:

```
pip install -e .
```

2.1 Documentation

To build the documentation (if using Sphinx) make sure you have all dependencies installed:

```
pip install Sphinx
pip install sphinx-autodoc-typehints
pip install myst-parser
pip install furo
```

```
cd documentation
make html
```

USER DOCUMENTATION

This chapter aims to help understand the Multi-body for Everybody 2D code usage. This is not the documentation for developers, but rather teaches how to define new examples to use the tool without knowledge of the math or what happens in the backend. What you can expect from this chapter is to learn how to define the multi-body system either using numeric values or symbolic values and how to loop over multiple symbolic parameters.

You will learn how to define simple bodies connected by Float, Revolut, or Prismatic joints. You will learn how to understand the visualization of your defined example, what are the degrees of freedom associated with each joint briefly and how to animate your simulation (the cool part).

3.1 How to Write an Example

The goal of the examples section is to build knowledge in a dynamic fashion. For that reason, we have decided to increase the explanations step-by-step and the amount of information each example contains increases as you progress in this tutorial.

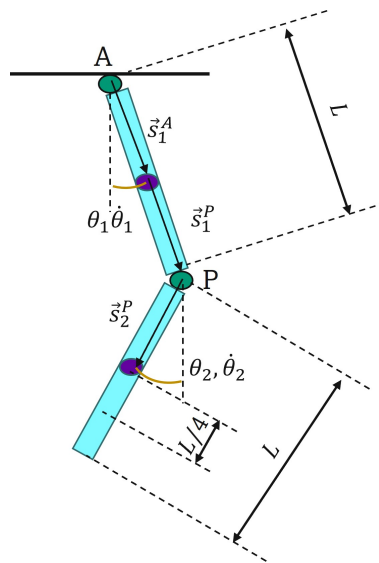
This section is organized into three main steps:

1. Quick examples overview explain how to set your example file without going into detailed explanation of what each variable means.
2. Detailed explanation examples, where bodies, points and forces definition are tackled in detailed.
3. Validation of the equations of motion (EOM) comparing results to Ginsberg book.

3.2 Quick examples overview:

3.2.1 Example 1: Double pendulum

The double-pendulum example is a multibody dynamics benchmark, which we use to validate the framework. For the analytical solution refer to Reference to theoretical paper. The following image represents the sketch of a double pendulum following the notation from the publication above. This is example number 3 from the default examples.



1 - Define origin of inertial frame of reference (f.o.r)

This section allows for origin offset, but how to use it is tackled in the next example.

Example 3

```
Reference_frame-Origin = np.array([0,0])
Reference_frame-Origin = -Reference_frame-Origin
```

2 - Bodies definition

a. Symbols for bodies (optional)

If you'll define symbolic body variables, initialize them first. Otherwise, you can leave this lists empty :

```
# Bodies definition
dataNames = [] # Define symbolic variables
BodyDataSym = symVars_definition(dataNames, globals())
```

b. Joint connectivity

Define lists for all the fields below:

```
joints          = [[0, 1], [1, 2]] # Joint connectivity: [parent,
child]
types           = ['R', 'R']       # Joint types: 'R' for revolute, 'P
→ ' for prismatic, 'F' for floating
parent_cg_to_joint = [[3, 0], [0, 1]] # Vectors from parent's center-of-
→ gravity (CG) to the joint location.
joint_to_child_cg  = [[0, 2], [0, 4]] # Vectors from the joint to the
child's CG.
prismatic_direction = [[np.nan, np.nan], [np.nan, np.nan]] # For prismatic
joints, the direction vector; for others, [nan, nan] is used.
prismatic_direction = normalize_prismatic(prismatic_direction)
```

3 - Points definition

This section allows to define additional points on top of the CG points and joints. For now we leave it blank. Even if not used, these fields must be declared empty.

```
# Points definition
PointNames = [] # Define symbolic variables for force points (for
example, b1, b2, b3).
PointsSym = symVars_definition(PointNames, globals())

# Define the structure for initial points.
Initial_Points = {}
# Ground points: these are fixed and given as [x, z] coordinates.
Initial_Points["GR"] = []

# Body-defined points ("BD"): we use a dictionary where each key (an
integer)
```

(continues on next page)

(continued from previous page)

```
# corresponds to a body and the value is a list of points.
Initial_Points["BD"] = {}
```

4 - Points definition

This section allows to define forces at specific location (CG, joints, additional points) and between bodies. For now we leave it blank. Even if not used, these fields must be declared empty.

```
# Forces definition
ForceNames = [] # Define symbolic variables for force points (for
example, b1, b2, b3).
ForceSym = symVars_definition(ForceNames, globals())

# Define the structure for Force.
Force = {}
# Forces applied on body-defined points (PointsBD):
Force["PointsBD"] = []

# Forces applied at the center of gravity (CG):
Force["CG"] = []

# Tension springs: stored as a list of tuples: (connection, [l0,
stiffness]).
# Here "BD41" means (for example) the 1st point on body 4.
Force["TensionSpring"] = []

# Tension dampers: here we have two pairs.
# The first pair uses a constant damping coefficient.
# The second uses a lambda function to represent a nonlinear damping
coefficient.
Force["TensionDamper"] = []

# Torsion springs: here the first element is a list of parameters and the
second element is a lambda.
Force["TorsionSpring"] = []
```

(continues on next page)

(continued from previous page)

```
# Torsion dampers:
Force["TorsionDamper"] = []

ForcesPointsSym = PointsSym + ForceSym
```

5 - Define numerical values

Now that joints, points, and forces are defined, you need to provide numerical values for:

- Initial conditions (positions/velocities)
- Mass & inertia of each body
- Gravity vector (percentage of “g” per body)
- Simulation time & timestep
- Animation options

a. Initial Conditions (ICs)

When bodies are initially defined, each body is given a body frame of reference which is parallel to the global frame of reference. The joint-coordinates (explained later) are initialized to zero at this initial configuration. This means that if we define our pendulum vertically it is not going to move, but otherwise it will and the initial conditions are defined as zero at that initial snapshot.

The second numerical values to initialized is `ForcesPointsNum` and `BodyDataNum`. These variables give a numerical quantity during time integration to quantities originally defined as symbolic. By default we always give it a value of 1 and they must always exist, even if unused.

```
# Initial conditions
# Create the JointSystem using the from_data class method.
joint_system      = JointSystem.from_data(joints, types, parent_cg_to_
→joint, joint_to_child_cg, prismatic_direction) # DO NOT MODIFY
Q, QD, _, NDOF, _ = joint_system.coordinate_finder() # DO NOT MODIFY

# If you are unsure what are the systems DOF run the example and you will
```

(continues on next page)

(continued from previous page)

```

see on screen
ic                                = np.zeros(2*sum(NDOF)) # Multiplied by 2 because is
position and velocity

# Parameters to loop over
ForcesPointsNum = np.ones(len(ForcesPointsSym))
BodyDataNum     = np.ones(len(BodyDataSym))

```

b. Time & Simulation parameters

Here we define values such as the mass and inertia of our bodies, the value of gravity (optional) and the percentage of gravity applied to each body (yes, not all bodies need to be subject to the same gravity value. Imaging a body is floating in water and others are not). We also define for how long we want to run our simulation and how big the time between integration steps will be.

```

# Simulation
TimeStep      = 0.05
tspan         = 5.
g             = 9.81 # gravity
gVec          = np.ones((len(types),1)) # Percentage of gravity acting on
each body
m0            = np.ones(len(types))
J0            = np.ones(len(types))

```

c. Animation parameters Very straight forward, do you want to visualize the animation? Yes (1) or no(0)?. Do you want to save it as gif? None for no and string name as 'name.gif' if you do. Lastly, is the animation too slow? Just speed it up by specifying how many time steps to skip (natural numbers only) with the variable `plotTstep`.

```

# Animation
animation_on   = 1 # Display animation:1, 0-otherwise
SaveMovieOn    = None # Extension:gif Saves animation if given a string
as name, otherwise set as None
plotTstep     = 5 # how many times faster are we plotting w.r.t to
simulation time

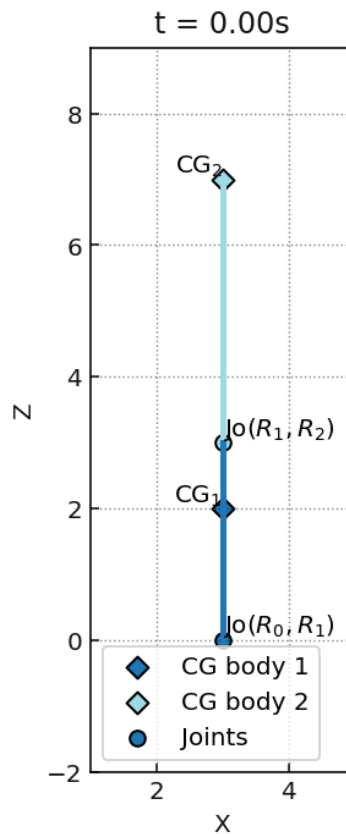
```

6 - call from main

Having defined the example it only remains importing it in the main file a running it. To import the spiderfloat example make sure the line in main where the Examples folders is read looks like this:

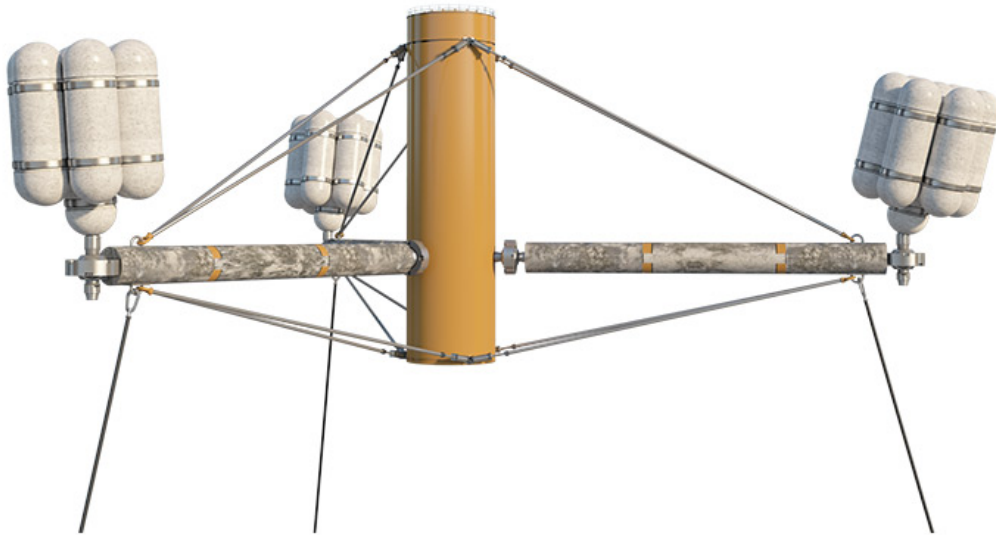
```
from Examples import Example3 as ex
```

Then just run the program! You should see a plot for the mechanical energy, which should be constant, and a plot with the initial snapshot. The initial snapshot should look like this:



3.2.2 Example 2: Spiderfloat

The Spiderfloat device (developed under ARPA-E ATLANTIS) is an offshore-wind platform designed to produce 1,200 GW of green energy. To see more, visit the [NREL Spiderfloat page](#). This example has reduced explanation on what each element to be defined is and focuses more on a general overview on how to set it up.



1 - Define origin of inertial frame of reference (f.o.r)

The user can choose the value of gravity and the origin of f.o.r which will influence how the bodies appear with respect to ground points. If the origin is $[0,0]$, and body 1 joint type is float, 'F', then the origin coincides with the first body CG. If `Reference_frame_Origin` is provided other than $[0,0]$, then the origin will be located at `Reference_frame_Origin` from the CG of body 1.

```
# Spiderfloat example
Reference_frame_Origin = np.array([-10,-5])
Reference_frame_Origin = -Reference_frame_Origin
```

2 - Bodies definition

a. Symbols for bodies (optional)

If you'll define symbolic body variables, initialize them first. Otherwise, you can leave this lists empty :

```
# Bodies definition
dataNames    = []      # Define symbolic variables
                        # for force points (for example, b1, b2, b3).
BodyDataSym = symVars_definition(dataNames, globals())
```

b. Joint connectivity

Define lists for:

- `joints`: each entry is `[parent_body, child_body]`
- `types`: corresponding joint type for each pair ('R', 'P', or 'F')
- `parent_cg_to_joint`: vector from parent's CG to the joint location
- `joint_to_child_cg`: vector from joint to the child's CG
- `prismatic_direction`: direction of translation for prismatic joints (set to `[np.nan, np.nan]` for 'R' or 'F' joints).

```
# (a) Joint connectivity: [parent, child]
joints = [
    [0, 1],
    [1, 2],
    [1, 3],
    [2, 4],
    [3, 5]
]

# (b) Joint types: 'R' = Revolute, 'P' = Prismatic, 'F' = Floating
types = ['F', 'R', 'R', 'R', 'R']

# (c) Parent.CG_to_Joint (meters)
parent_cg_to_joint = [
    [10, 10],      # Parent = 0 to Joint 0
    [0.5, -2],     # Parent = 1 to Joint 1
    [-0.5, -2],    # Parent = 1 to Joint 2
    [4.5, 0],      # Parent = 2 to Joint 3
    [-4.5, 0]      # Parent = 3 to Joint 4
]

# (d) Joint_to_Child.CG (meters)
joint_to_child_cg = [
    [np.nan, np.nan], # For 'F' joint, use [nan, nan]
    [5, 0],           # Joint 1 to Child 2's CG
    [-5, 0],          # Joint 2 to Child 3's CG
    [0, 1.5],         # Joint 3 to Child 4's CG
]
```

(continues on next page)

(continued from previous page)

```

    [0, 1.5]          # Joint 4 to Child 5's CG
]

# (e) Prismatic direction vectors (only for 'P' joints; else use [nan,
nan])
prismatic_direction = [
    [np.nan, np.nan],
    [np.nan, np.nan],
    [np.nan, np.nan],
    [np.nan, np.nan],
    [np.nan, np.nan]
]

# Normalize the prismatic direction (no effect on [nan, nan])
prismatic_direction = normalize_prismatic(prismatic_direction)

```

Notes:

- *Order matters. Parent bodies should be already defined for every new joints you create, for example, if you haven't defined body 3 before, you can't say body 3 is parent of body 4
- For floating joints ('F'), set their geometry vectors to [nan, nan].

3 - Points definition

Points let you apply forces at specific locations. There are two categories:

- Ground points ("GR"): fixed in world coordinates
- Body-defined points ("BD"): move with their respective body

Initialize the point dictionaries `Initial_Points` (even if you don't use them):

```

# Symbolic point names (if needed):
PointNames = []
PointsSym = symVars_definition(PointNames, globals())

# Initialize containers

```

(continues on next page)

(continued from previous page)

```
Initial_Points = {}
Initial_Points["GR"] = []    # Ground points (list of [x, y] coordinates)
Initial_Points["BD"] = {}    # Body points (dict: body_index + list of [x, y] for that body)
```

a. Define ground points

```
# Example: two ground points at (20, 0) and (0, 0)
Initial_Points["GR"] = [
    [20, 0],
    [0, 0]
]
```

b. Define body points

```
# Example: Body 1 has four points; Body 2 has two; Body 3 has two
Initial_Points["BD"][1] = [
    [0.5, -2.5],
    [-0.5, -2.5],
    [0.5, 5],
    [-0.5, 5]
]
Initial_Points["BD"][2] = [
    [4, 0],
    [4.5, 0]
]
Initial_Points["BD"][3] = [
    [-4, 0],
    [-4.5, 0]
]
```

Remember: Even if you don't need ground points or body points, you must still initialize:

```
Initial_Points["GR"] = []
Initial_Points["BD"] = {}
```

4 - Forces definition

Initialize symbolic force names (if needed):

```
ForceNames = []
ForceSym    = symVars_definition(ForceNames, globals())

Force = {}
```

Then define each force category. Even if a category has no entries, assign an empty list so that the “checks” run consistently (especially if you’re converting from MATLAB, which might implicitly remove empty variables).

a. Point forces applied to the body-defined points

```
# Each entry: [body_index, point_index, Fx, Fz, My]
Force["PointsBD"] = [
    # Example: apply a moment of 1 at Body 4's 1st point [4, 0, 0, 0, 1]
    (no forces here for Spiderfloat)
]
```

b. Forces applied to the body centers of gravity (CG)

```
# Each entry: [body_index, Fx, Fz, My]
Force["CG"] = [
    # Example: [2, 5, 0, 0] applies Fx=5 N on Body 2's CG
]
```

c. Tension springs

```
# Each entry is ((pointA, pointB), [l0, stiffness])
# pointA/pointB are strings like "BD10" = Body 1 Point 0, "GR01" = Ground
Point 1
Force["TensionSpring"] = [
    (("BD10", "BD20"), [7.0139, 1e0]),
    (("BD11", "BD30"), [7.0139, 1e0]),
    (("BD12", "BD21"), [11.8004, 1e-1]),
    (("BD13", "BD31"), [11.8004, 1e-1]),
    (("BD20", "GR00"), [3.0414, 1e-1]),
]
```

(continues on next page)

(continued from previous page)

```

    (("GR01", "BD30"), [3.0414, 1e-1])
]

```

d. Tension dampers

```

# Each entry: ((pointA, pointB), damping_coefficient)
Force["TensionDamper"] = [
    (("GR00", "CG22"), 3),
    (("GR01", "CG33"), 3)
]

```

e. Torsion springs & dampers

If you have torsion springs or torsion dampers, add them—even if empty—to keep the structure consistent:

```

Force["TorsionSpring"] = [
    # ((bodyA, bodyB), [theta0, k_theta])
]

Force["TorsionDamper"] = [
    # ((bodyA, bodyB), c)
]

```

Key point:

- `PointsBD` and `CG` use lists of lists (each inner list = one force point).
- Springs and dampers use lists of tuples.

5 - Define numerical values

Now that joints, points, and forces are defined, you need to provide numerical values for:

- Initial conditions (positions/velocities)
- Mass & inertia of each body
- Gravity vector (percentage of “g” per body)
- Simulation time & timestep

- Animation options

a. Initial Conditions (ICs)

You can either manually specify every initial position and velocity if you are already familiar with joint coordinates, or use `coordinate_finder()` to find all the positions and velocity that must be initialized.

```
# Build the JointSystem (DO NOT modify)
joint_system = JointSystem.from_data(
    joints,
    types,
    parent_cg_to_joint,
    joint_to_child_cg,
    prismatic_direction
)

# Extract coordinate info (DO NOT modify)
Q, QD, _, NDOF, _ = joint_system.coordinate_finder()
```

- NDOF is an array that gives the number of degrees of freedom for each joint.
- The total number of position variables is `sum(NDOF)`.
- Because we need both positions and velocities, the total IC length is `2 * sum(NDOF)`.

If you're unsure, initialize everything to zero:

```
# Zero initial conditions (positions + velocities)
ic = np.zeros(2 * sum(NDOF))
```

You can then modify `ic[...]` if you want specific starting angles/velocities.

b. Symbolic Parameters to Loop Over (optional)

If you want to explore multiple force or body parameters, define arrays of symbolic values:

```
# For example, if ForceSym and BodyDataSym are nonempty:
ForcesPointsNum = np.ones(len(ForceSym)) # Start with ones
BodyDataNum      = np.ones(len(BodyDataSym)) # (modify later if needed)
```

At the moment, we initialize them to ones (no effect). You can expand this later to loop over parameter sets. This portion is still unfinished

c. Time & Simulation Parameter

```
TimeStep = 0.05      # [s] integration interval
tspan    = 10.0      # [s] total simulation time (final time)
```

d. Gravity per Body

- `g`: global gravity constant (e.g., 9.81)
- `gVec`: an array of size (`n_bodies`, 1) specifying the percentage of gravity each body experiences. For example, if you want zero gravity on Body 2, set `gVec[1] = 0.0`.

```
g      = 9.81          # standard gravity [m/s²]
gVec = np.ones((len(types), 1)) # 100% gravity on each body
```

e. Mass & Inertia

Provide a 0-D NumPy array (i.e., a 1D array) for each:

- `m0`: mass of each body
- `J0`: moment of inertia of each body (about its CG)

```
m0 = np.ones(len(types)) # Example: all masses = 1 kg
J0 = 2.0 * np.ones(len(types)) # Example: all inertias = 2 kg·m²
```

f. Animation Options

These flags control whether you see an on-screen animation and/or save a GIF:

- `animation_on`: 1 to display, 0 to skip
- `SaveMovieOn`: set to `None` (no file) or e.g. `"Spiderfloat.gif"`
- `plotTstep`: draw every `plotTstep`-th frame (for speed)

```
animation_on = 1      # 1 = show animation; 0 = skip
SaveMovieOn  = None   # "Spiderfloat.gif" to save, or None to skip
plotTstep    = 5      # Plot 1 frame every 5 simulation frames
```

6 - call from main

Having defined the example it only remains importing it in the main file a running it. To import the spiderfloat example make sure the line in main where the Examples folders is read looks like this:

```
from Examples import spiderfloat as ex
```

Then the multibody system is created with the lines:

```
# 1- Initialize the Multibody system  
MBDsys = MBDSysSystem.from_example(ex)
```

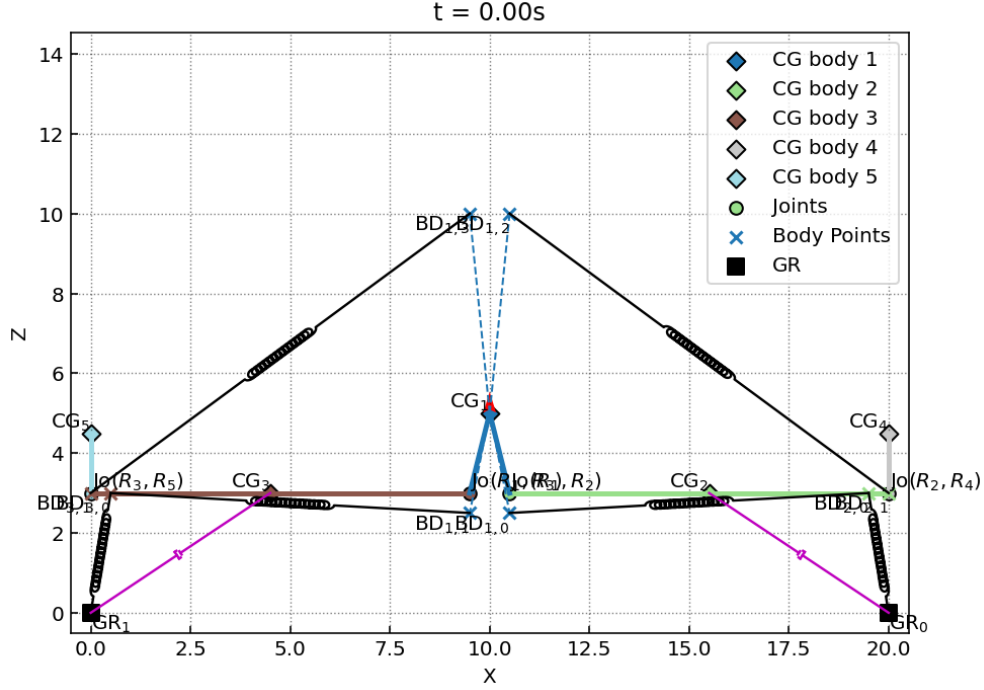
The next step is to integrate over time. For that we need to define our variable containing all numeric values as:

```
# 3 - Define initial numerical values  
mainNumVars = np.hstack((ex.ic, ex.ForcesPointsNum, ex.BodyDataNum))
```

Then we can integrate the system

```
# 3 - Integrate over time  
sol      = MBDsys.integrate(mainNumVars, ex.m0, ex.J0,  
                             tspan=ex.tspan, dt=ex.TimeStep, external_  
→manager=None)
```

If everything went correctly, your first frame should look like this:



3.3 Detailed examples

3.3.1 Example 1: FOSWEC

To model a multibody system, you need to define a kinematic sketch of the system at its initial condition. These are the steps we use to create the sketch of the system:

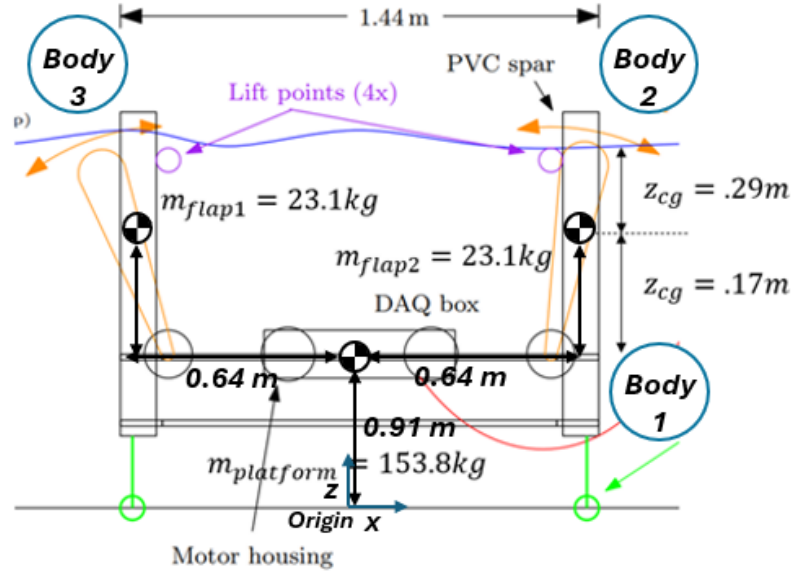
1. Defining the joints of the system (starting with joint 1 which always connects body 1 to ground via prismatic, revolute, or floating joint).
2. Each joint has a parent and a child (for instance for joint 1, parent is always the ground and its child is always body 1). Then, for each joint, we define two vector: one vector connects the joint location to parent CG and the other vector connects the joint to child's CG. If we do this for all the joints, we have defined a kinematic sketch of the system.

There are a few exceptions:

- For instance, for joint 1, the parent is body 0 (ground) and it doesn't have a CG.
- CG of ground is the inertial frame. Basically, the parentCG to joint 1 shows the location of joint 1.

This example shows how the FOSWEC dynamics model can be created using our proposed

code and how correct implementation is verified. Additionally we show the joints DOFs associated to the example. The equations of motion are provided symbolically using joint coordinates (reduced coordinates).



1 - Define origin of inertial frame of reference (f.o.r)

This time we define the origin of the frame of reference 0.91 units underneath the center of mass of body 1.

```
Reference_frame_Origin = np.array([0,-0.91])
Reference_frame_Origin = -Reference_frame_Origin
```

2 - Bodies definition

This time we are presenting the result we achieve to aim when defining the bodies in table format before actual definition, and if the user feels comfortable it is recommended that you try it out before seeing the solution.

Bodies and joints table:

Joint	Connected Bodies	Joint Type	ParentCGtoJoint	JointtoChildCG	Prismatic	Direction
1	0 1	F	[0, 0]	NaN NaN		NaN NaN
2	1 2	R	[0.640000000000000, 0]	[0, 0.170000000000000]		NaN NaN
3	1 3	R	[-0.640000000000000, 0]	[0, 0.170000000000000]		NaN NaN

This table is automatically generated when running the example FOSWEC.py either on its own or from main. To obtain such table we must define the bodies as follows

```
joints          = [[0, 1],[1, 2],[1,3]]
types           = ['F', 'R', 'R']
parent_cg_to_joint = [[0, 0],[0.64, 0],[-0.64, 0]]
joint_to_child_cg  = [[np.nan, np.nan],[0, 0.17],[0, 0.17]]
prismatic_direction = [[np.nan, np.nan],[np.nan, np.nan],[np.nan, np.nan]]
prismatic_direction = normalize_prismatic(prismatic_direction)
```

The key point between the bodies definition and how it is displayed as table is that one is the transposed of the other. This means that if we read the information of the code vertically, i.e. column by column, that information will correspond to each row of the table.

Each row of this table represents a kinematic joint. The entire kinematic structure is built by defining the joints and their associated vectors. The types of joints that can be modeled include: Floating Joint (F), Prismatic Joint (P), and Revolute Joint (R).

Let's go over the first row. Each joint has a parent body and a child body. Each row defines how the joint is connected to the child's center of gravity (CG) and the parent's CG. This is how the entire kinematic structure is defined.

- The first column indicates the parent of the joint, and the second column indicates the child.
- The parent of the first defined joint is always set to zero, which represents the ground. The child body, in this case, is body 1.
- The third column specifies the joint type. For body 1, which is connected to the ground, this is a floating joint (representing the platform's base).
- The vectors in the fourth and fifth columns represent the [x,z] components of the vector from the ground (reference frame) to the CG of the floating body.
- The sixth and seventh columns should be set to [NaN,NaN] for floating joints.
- The last two columns are used only for prismatic joints to define the initial direction of motion and should be set to [NaN,NaN] for all non-prismatic joints.

The second row of the table indicates that a revolute joint connects body 1 (the parent) to body 2 (the child). Note that body 1 was already defined in the first row.

- The third and fourth columns specify the vector from the CG of body 1 to the revolute

joint. This joint is located at $[0.64, 0]$ meters relative to the CG of body 1.

- The fifth and sixth columns specify the vector from the revolute joint to the CG of body 2, which is $[0, 0.17]$.

In summary, this row defines the connection between the platform's base and the right flap.

The third row indicates that a revolute joint connects body 1 (the parent) to body 3 (the child).

- The third and fourth columns define the vector from the CG of body 1 to the revolute joint, which is located at $[-0.64, 0]$ meters.
- The fifth and sixth columns define the vector from the revolute joint to the CG of body 3, which is $[0, 0.17]$.

In summary, this row adds the left flap to the platform's base.

3 - Joint coordinates DOFs

The joint coordinates associated to this example are extracted at the initial conditions section, but for the sake of explaining a bit the coordinates associated with each type of joint we moved it here.

```
Q, QD, _, NDOF, _ = joint_system.coordinate_finder()
```

Running this line requires creating the `joint_system` but that is already in the line above in the code and it's not important at the moment. If we print on screen these coordinates we get a set of degrees of freedom with repeated subindices, sometimes. Each subindex, points to a different body. Therefore if a subindex is repeated it means that the joint type has multiple DOFs associated. For our example it looks like this:

```
Q' = [ X_1, Z_1, Theta_1, Theta_2, Theta_3]
```

There are three subindices: 1, 2, 3. This means that we have three bodies. The first body was defined as a floating body or joint. Floating joints have in 2D three degrees of freedom, two translations: X, Z and one rotation θ . The second and third bodies are connected to the first one through revolute joints, and this joint has one rotational degree of freedom: θ . So in summary, $[X_1, Z_1]$ show the position of the CG of the platform and $[\theta_1, \theta_2, \theta_3]$ show the pitch angle of the platform, the first flap, and the second flap, respectively.

4 - Points definition

To define springs and dampers associated to the the bodies, unless these points are either joints or the center of gravity, we must define them on top of the bodies definition. As mentioned in the previous example, if the bodies are fixed we define them as ground points, **GR**, whereas if this points are on a body, we define them as body points, **BD**. The definition of the points for FOSWEC is summarized in the table below and is used to verify we have defined everything as we wanted.

Points table:

Body	PointType	PointID	Rel coordinates
0	GR	0	[0.75, 0]
0	GR	1	[-0.75, 0]
1	BD	0	[0.75, 0]
1	BD	1	[-0.75, 0]

- Column 1: body to which the point belongs. Ground is always associated to 0.
- Column 2: Point type
- Column 3: Order in which the point was defined in a body
- Column 4: x-coordinate relative to the CG
- Column 5: y-coordinate relative to the CG

This points are defined in using the following lines of code:

```
# Ground or fixed points
Initial_Points["GR"] = [[0.75, 0],[-0.75, 0]]
# Body associated points
Initial_Points["BD"] = {}
Initial_Points["BD"][1] = [[0.75, 0],[-0.75, 0]]
```

The variable `Initial_Points` is a dictionary with two entried:

- **GR** for ground points
- **BD** for body points

The entry **GR** is a list containing the absolute coordinates of each point. On the other hand, the entry **BD** is another dictionary, which entries refer to the additional points each body has. Each dictionary entry, contains a list of points. In this example, only body 1 has associated points and it has two points. The coordinates entered for the `Initial_Points['BD']` are

relative to the CG. Thus the first point in body 1 is located 0.75 units to the right of the body 1 CG.

After defining these popints, we can access them at the code. For instance point `[0.75, 0]`, the first point defined in body 1 is accessed as `'BD10'` and point `[-0.75,0]` defined on body 1 also is accessed as `'BD11'`

5 - Forces definition

For the forces definition section, we are going to introduce a new concept, defining symbolic forces. To define symbolic variables in section, we must define the variables as a list of strings in their corresponding section. Note: if a variable is repeated define only once. In this example we define the time as symbolic variable, `t`, and the springs stiffness, `k`. The string name is defined under the variable `ForceNames` and the function `symVars_definition` converts those names into variables and places them in the global workspace.

```
ForceNames = ['t','k']
ForceSym    = symVars_definition(ForceNames,globals())
```

The final result will look like this:

```
Forces table:
ForceType BodiesConnected PointType PointID ForceComponents SpringParameters DamperParameters
CG 1 CG NA [0, 0.1*exp(-t), 0] None NaN
TensionSpring 0,1 GR,BD 0,0 None [5, k] NaN
TensionSpring 0,1 GR,BD 1,1 None [5, k] NaN
TensionDamper 0,1 GR,BD 0,0 None None 1.0
TensionDamper 0,1 GR,BD 1,1 None None 1.0
TorsionSpring 1,2 JO NA None [0, 10] NaN
TorsionSpring 1,3 JO NA None [0, 10] NaN
TorsionDamper 1,2 JO NA None None 0.0
TorsionDamper 1,3 JO NA None None 0.0
```

The explanation for each of the entries is divided into two groups. One point forces PointsBD and CG which are defined as lists of lists and the two points forces which are defined as list of tuples.

- PointsBD: [lists of lists] where each list corresponding to one point contains
 1. Body of application i , which corresponds to the dictionary entry in `Initial_points`.
 2. Element in the list j of `Initial_points[i]`.
 3. F_x : force in x-direction
 4. F_z : force in z-direction

5. M_y : moment in y-direction

In this example PointsBD is defined as empty, but must always be defined for consistency with MATLAB code.

```
Force["PointsBD"] = []
```

- CG: [lists of lists] where each list corresponding to one point contains
 1. Body of application i, which corresponds to the dictionary entry in Initial_points.
 2. F_x : force in x-direction
 3. F_z : force in z-direction
 4. M_y : moment in y-direction

With the time variable we are going to define an exponential force at the CG of body 1, with only vertical component. This force is an exponential decay function $F_y = e^{-t}$.

```
Force["CG"] = [[1,0,0.1*sym.exp(-t),0]]
```

- TensionSpring: list of tuples where each tuple contains
 1. Tuple of strings that refer to the two points connected by the spring
 2. List of parameters containing l_0 : undeformed length, k : stiffness constant if linear spring (will be clarified later)]

In this example we are connecting the first ground point, 'GR00', with the first point in body 1, 'BD10'. These points were already defined in the dictionary Initial_Points. Then we define the unstretched length, l_0 as 5, and the stiffness, k , will be symbolic.

```
Force["TensionSpring"] = [((('GR00', 'BD10'), [5,k]),
                             (('GR01', 'BD11'), [5,k])
                             )]
```

- TensionDamper: list of tuples where each tuple contains
 1. Tuple of strings that refer to the two points connected by the damper
 2. float containing c : damping coefficient if linear spring (will be clarified later)

Tension dampers are defined in a very similar manner to tension springs. The points are defined in the same manner, however, when defining the parameters, instead of passing a

list we pass a float corresponding to the damping coefficient.

```
Force["TensionDamper"] = [ (('GR00','BD10'),1),
                           (('GR01','BD11'),1)]
```

- TorsionSpring: list of tuples where each tuple contains
 1. Tuple of strings or floats that refer to the two bodies connected by the damper
 2. list containing $[\theta_0$: undeformed angle, k_θ : stiffness]

Torsion springs follow the same structure as Tension Springs, however, instead of defining the points connected we defined the bodies connected as this springs are only conceived to be at the joints between bodies. These points can be defined as a **tuple** of two **floats** or two **strings**. The parameters are defined in the same fashion as tension springs.

```
Force["TorsionSpring"] = [ ((1,2),[0,10]),
                           ((1,3),[0,10])]
```

- TorsionDamper: list of tuples where each tuple contains
 1. Tuple of strings or floats that refer to the two bodies connected by the damper
 2. float containing c : damping coefficient if linear spring (will be clarified later)

Torsion dampers follow the same structure as Torsion springs to define the points and the same structure as tension dampers to define the damping coefficient.

```
Force["TorsionDamper"] = [ ((1,2),0),
                           ((1,3),0)]
```

6 - Simulation parameters

This step is needed to simulate the system over time. The parameters defined before, could have been symbolic or numeric, to simulate the system over time, you need numerical values.

In this section we must provide at least the time parameters and the mass and inertia properties. The gravity by default is set to 9.81 and applied to all bodies. We define the parameters in the following manner:

- TimeStep: specifies the integration time taken in forward integration
- tspan: defines the end-time of the integration

- `g`: it is the value of gravity's acceleration
- `gVec`: it is a vector that defines, for each body, a gravity multiplier, i.e.: a value of 2 in the first position would indicate that the first body is subject to 2 times the value of `g`.
- `m0`: it is a 0-D array specifying the mass of each body
- `J0`: it is a 0-D array specifying the second moment of inertia of each body

```
# Simulation
TimeStep      = 0.05
tspan         = 10.
g             = 0.                # gravity
gVec          = np.ones((len(types),1)) # Percentage of gravity acting on
each body
m0            = np.ones(len(types))
J0            = 2.*np.ones(len(types))
```

The animation parameters are not related to the integration but rather just visualization. They define:

- `animation_on`: whether we want to display animation or not. It is a boolean and takes values of 0 or 1.
- `SaveMovieOn`: if name is specified with `.gif` extension, it saves the animation in the root folder.
- `plotTstep`: it is used to speed up the animation and specifies every how many time steps to plot. It must be an integer.

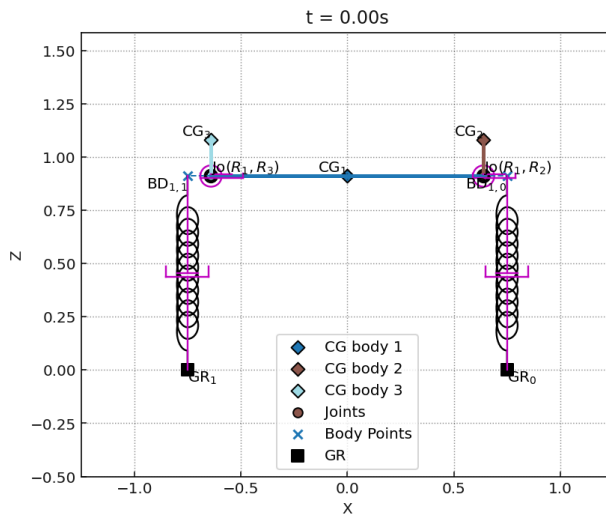
```
# Animation
animation_on   = 1                # Display animation:1, 0-otherwise
SaveMovieOn    = None            # Extension:gif Saves animation if given a
string as name, otherwise set as None
plotTstep      = 5                # how many times faster are we plotting w.r.t
to simulation time
```

7 - Simulation and results

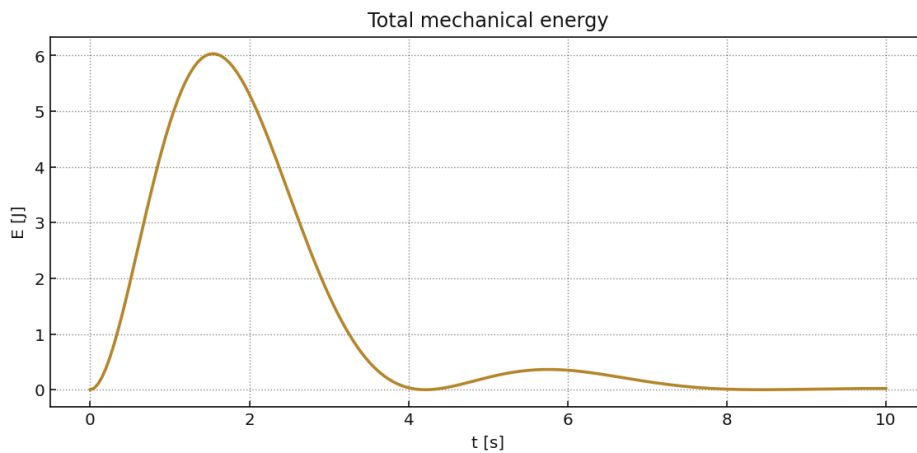
To run this example you only need to update one line in the main file. To import the FOSWEC example it must look like:

```
from Examples import FOSWEC as ex
```

Foswec initial configuration looks like this (set `tspan=0.05` to get this visualization):



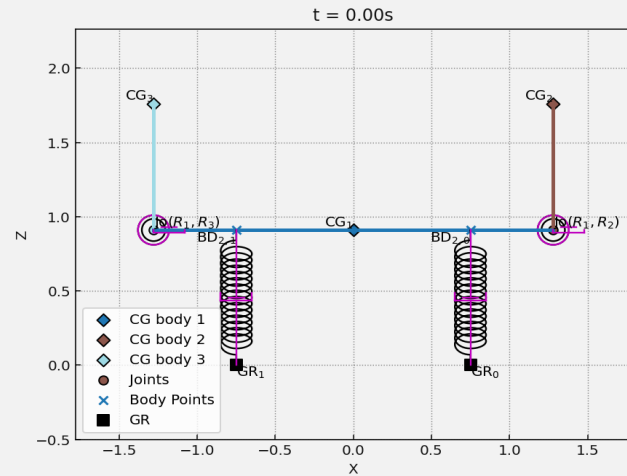
and the energy plot looks:



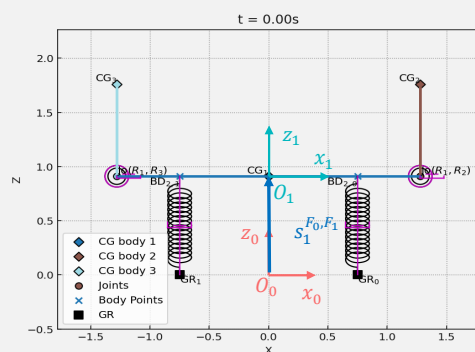
Step-by-step PPT

This portion contains the slides presenting this example. The slides with the animations are in TODO: ref to slides.

FOSWEC



Bodies definition: body 1



Connecting bodies

Joint type

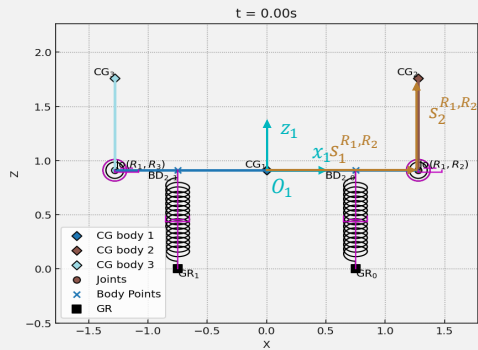
```
joints = [[0, 1], [1, 2], [1, 3]]
types = ['F', 'R', 'R']
parent_cg_to_joint = [[0, 0.91], [2*0.64, 0], [-2*0.64, 0]]
joint_to_child_cg = [[np.nan, np.nan], [0, 5*0.17], [0, 5*0.17]]
prismatic_direction = [[np.nan, np.nan], [np.nan, np.nan], [np.nan, np.nan]]
prismatic_direction = normalize_prismatic_directions(prismatic_direction)
```

$s_0^{F_0, F_1}$
'F' joints only
Parent cg to joint

Direction of allowed translation

Go to next slide to continue animation

Bodies definition: body 2



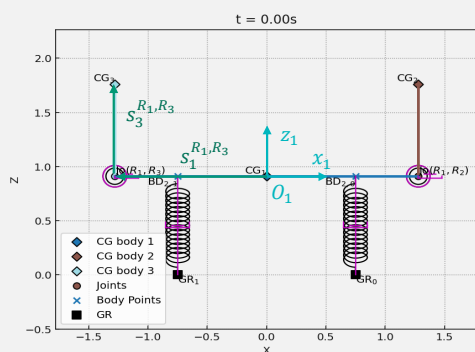
Connecting bodies

Joint type

```
joints = [[0, 1], [1, 2], [1, 3]]
types = ['F', 'R', 'R']
parent_cg_to_joint = [[0, 0.91], [2*0.64, 0], [-2*0.64, 0]]
joint_to_child_cg = [[np.nan, np.nan], [0, 5*0.17], [0, 5*0.17]]
prismatic_direction = [[np.nan, np.nan], [np.nan, np.nan], [np.nan, np.nan]]
prismatic_direction = normalize_prismatic_directions(prismatic_direction)
```

Go to next slide to continue animation

Bodies definition: body 3



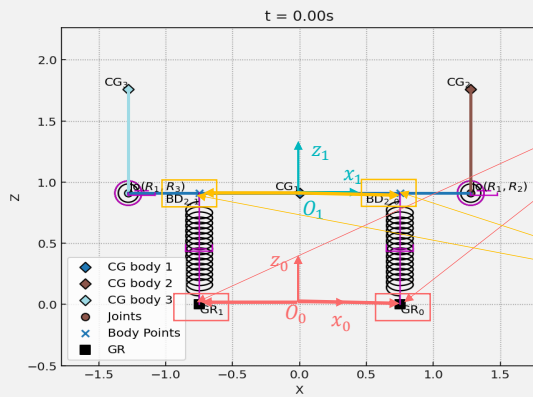
Connecting bodies

Joint type

```
joints = [[0, 1], [1, 2], [1, 3]]
types = ['F', 'R', 'R']
parent_cg_to_joint = [[0, 0.91], [2*0.64, 0], [-2*0.64, 0]]
joint_to_child_cg = [[np.nan, np.nan], [0, 5*0.17], [0, 5*0.17]]
prismatic_direction = [[np.nan, np.nan], [np.nan, np.nan], [np.nan, np.nan]]
prismatic_direction = normalize_prismatic_directions(prismatic_direction)
```

Go to next slide to continue animation

Points definition

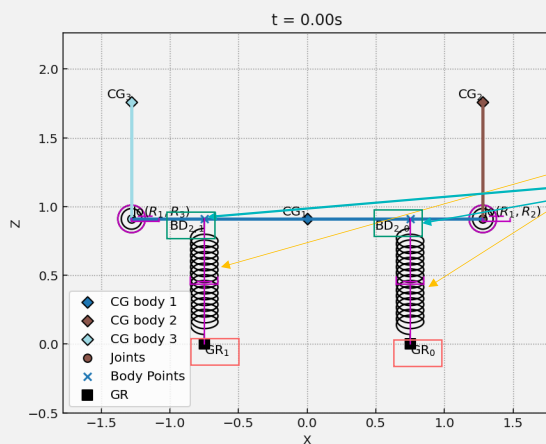


`Initial_Points["GR"] = [[0.75, 0], [-0.75, 0]]`

`Initial_Points["BD"] = {}`
`Initial_Points["BD"][1] = [[0.75, 0], [-0.75, 0]]`

Go to next slide to continue animation

Forces definition: tension springs



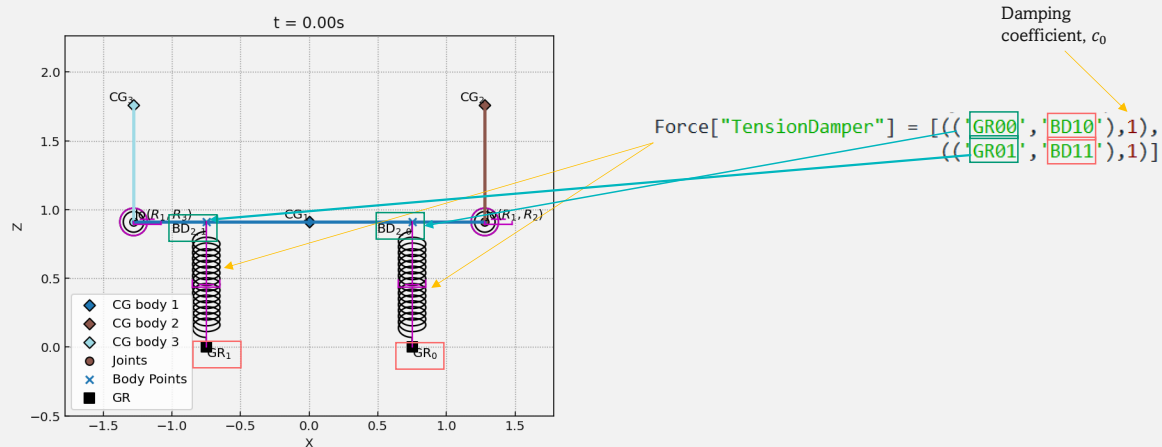
`Force["TensionSpring"] = [((['GR00'], ['BD10']), [5, k]), ((['GR01'], ['BD11']), [5, k])]`

Undeformed length, l_0

Stiffness, k

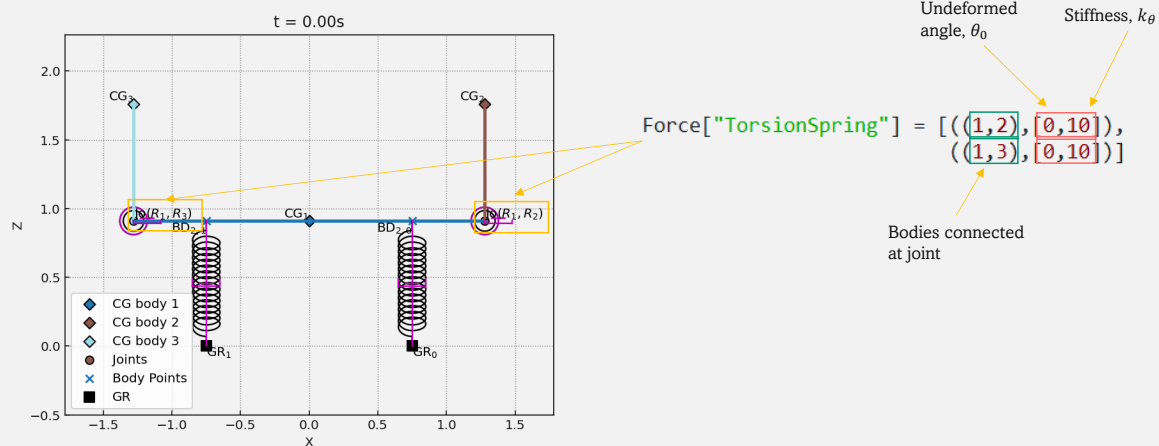
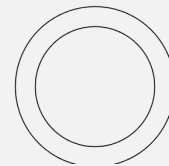
Go to next slide to continue animation

Forces definition: tension dampers



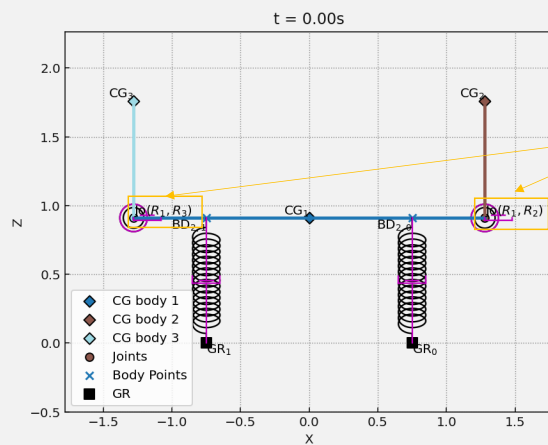
Go to next slide to continue animation

Forces definition: torsion springs



Go to next slide to continue animation

Forces definition: torsion dampers



Force["TorsionDamper"] = [((1,2),0),
((1,3),0)]

Damping
coefficient, c_θ

Bodies connected
at joint

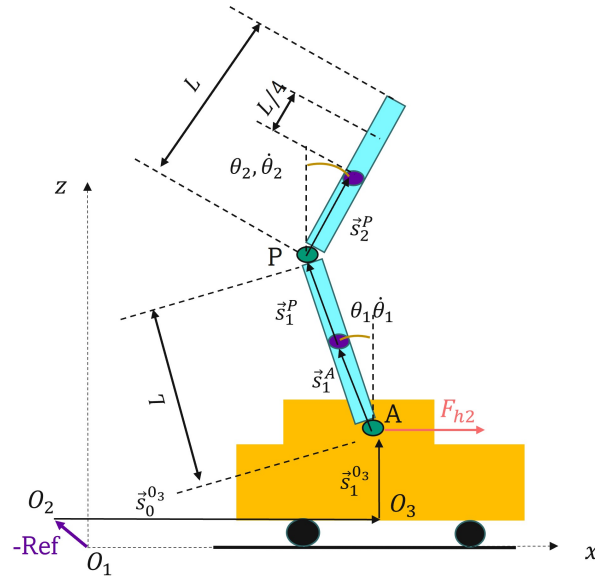
Go to next slide to continue animation

3.3.2 Example 2: Double pendulum on cart

In this example we want to model a well known example in multibody dynamics and now if interest in the machine learning community, double pendulum on a cart. This example is interesting from the control theory perspective, although we will not focus on stabilizing the pendulum on top of the cart but rather visualize the motion. For you is interesting also because you will learn how to model prismatic joints and a dig deeper in the origing offset. Additionally, you will define time dependent forces at the center of mass. This is example 4 of the default examples, except for the displace origin.

1 - Define origin of inertial frame of reference (f.o.r)

We want to model the base of the body to be above the ground 1 unit. One way to do so without over-complicating the vectors `parent_cg_to_joint` and `joint_to_child_cg` is to offset where the bodies definition starts. The way this offset works is a vector that goes from the joint of body 1 that connects to the ground to the origin, i.e. the opposite of the vector you want (This feature is being investigated and may be changed in the future). This feature may be useful when defining the prismatic direction as horizontal but we want some offset from the ground, if anything is parametric.



```
Reference_frame_Origin = np.array([1,-1])
Reference_frame_Origin = -Reference_frame_Origin
```

Check the schematics to understand better what `Reference_frame_Origin` means. After the body definition, we will clarify a few points on what it means then to define

`parent_cg_to_joint` for the bodies in contact with ground and `joint_to_child_cg`.

```
joints          = [[0, 1],[1, 2],[2,3]]
types           = ['P','R', 'R']
parent_cg_to_joint = [[3, 0],[0,0],[0,1]]
joint_to_child_cg  = [[0,1],[0, 2],[3,0]]
prismatic_direction = [[3, 0],[np.nan, np.nan],[np.nan, np.nan]]
prismatic_direction = normalize_prismatic(prismatic_direction)
```

By moving the reference frame to $[-1,1]$ (The opposite of what we wrote), then we transformed `parent_cg_to_joint`, vector $\vec{s}_0^{O_3}$, to a horizontal line defined by vector $[3,0]$. If we want to define this vector symbolically and then the prismatic direction for the cart as the same vector, offsetting the origin becomes handy.

Note that we have defined the first body as prismatic joint type. This means that we must define a prismatic direction associated to it. We can define it with whatever vector we want but then its norm is normalized using the function `normalize_prismatic` which lives under `multibody`.

Another interesting feature of prismatic joints is that they do not possess their own rotational degree of freedom, and therefore inherit the rotation of the previous body in the chain. In this case the previous body is the ground, hence the cart has no rotation at all and will move only horizontally.

In summary, this system has three bodies connected by prismatic and revolut joint. These joints contribute with 1 translational DOF (prismatic joint) and 2 rotational DOFs (1 per revolut joint). We name the translational DOFs with **S** and the rotational DOFs with **Theta**. Remember that the subindex shows which body they belong to. We can print the DOFs on screen by running the example script by itself. For a closer look you can check the following lines:

```
Q, QD, QDD, NDOF, _ = joint_system.coordinate_finder()
```

These variables will show you the positional DOFs, **Q**, as well as the velocity and acceleration coordinates, **QD**, **QDD**, respectively. Lastly, **NDOF** summarizes the number of DOFs per joint, so in this case it would be a list showing $[1,1,1]$.

2 - Points definition

This system does not have forces applied outside from the CG, thus no point definition is required. However, the list for GR points and the dictionary for BD points must be initialized.

```
Initial_Points["GR"]    = []
Initial_Points["BD"]    = {}
```

3 - Forces definition

The forces definition involves the last learning point of this example. Define a time-dependant force at the CG of body 1. To define a time-dependant force, we must initialize it as a symbolic variable. Since we are using it in the force section we initialize it in this section. If it was used in multiple sections, it should only be initialized in the FIRST one. To define this symbolic variable, we define a string "t" in the list `ForceNames`.

```
ForceNames = ["t"]
ForceSym    = symVars_definition(ForceNames, globals()) # DO NOT MODIFY
```

The second line, where `ForceSym` is defined, and which should not be touched, converts each element in the list into a `sympy Symbol` with the assumption of `real=True`. This is important because manipulation of this variable will involves taking into account that the variable is real. Check [SymPy Gotchas](#) to dig deeper into the concept.

Once the symbolic variables needed are defined, then we can proceed to define the force of interest. Remember all the force types must be defined, even as empty. We are interested in a force applied on the CG of body 1. Then, in the force dictionary, in the key: "CG" we can create a list for each for applied in the CG. The elements on the list will specify to which body is the force applied to and the components of the force as `Fx`, `Fz`, `My`. In our case, we decided to apply a horizontal force with exponential decay. Note that this definitions must follow `SymPy` syntax.

The force definition lines look like this:

```
Force["PointsBD"]      = []
Force["CG"]             = [[1, sym.exp(-t), 0, 0]]
Force["TensionSpring"] = []
Force["TensionDamper"]  = []
Force["TorsionSpring"] = []
```

(continues on next page)

(continued from previous page)

```
Force["TorsionDamper"] = []

ForcesPointsSym = PointsSym + ForceSym
```

The last line in the code above groups all the symbolic variables defined in the points and forces section to create the numeric association and mapping latter on. It must exists but it is not of the users concern. It appears here cos it is necessary for the checks.

The remaining lines for numerical values and simulation are left for completeness but it requires no user input and they have been already explained.

4 - Numerical simulation and animation

Just a quick comment, we saw that this problem has 3 DOFs, one translational and two rotationals. Since we are solving a 2nd order ODE, it requires initial conditions for position and velocity, thus double of the number of DOFs.

```
# If you are unsure what are the systems DOF run the example and you will
see on screen
ic = np.array([0,0,0,0,0,0]) # Multiplied by 2 because is
position and velocity

# Parameters to loop over
ForcesPointsNum = np.ones(len(ForcesPointsSym))
BodyDataNum = np.ones(len(BodyDataSym))

# Simulation
TimeStep = 0.05
tspan = 5.
g = 0. # gravity
gVec = np.ones((len(types),1)) # Percentage of gravity acting on
each body
m0 = np.ones(len(types))
J0 = 2.*np.ones(len(types))

# Animation
animation_on = 0 # Display animation:1, 0-otherwise
```

(continues on next page)

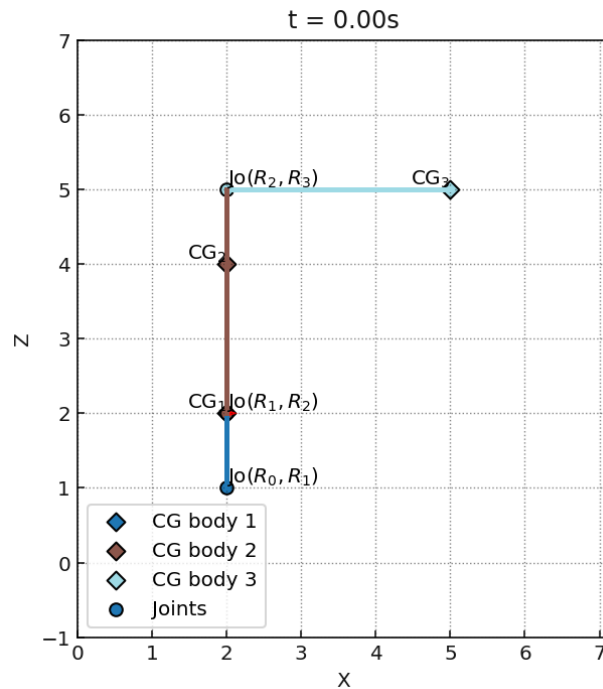
(continued from previous page)

```

SaveMovieOn      = None      # Extension:gif Saves animation if given a
                               # string as name, otherwise set as None
plotTstep        = 5         # how many times faster are we plotting w.r.t
                               # to simulation time

```

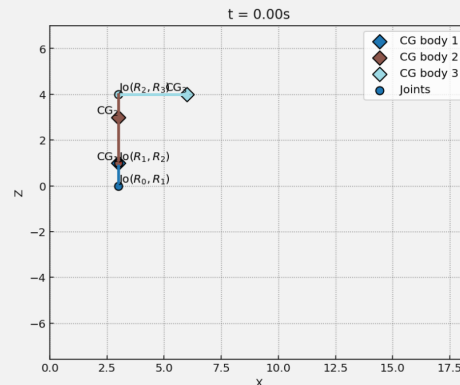
And after all that hard effort ... this is how it looks:



Step-by-step PPT

This portion contains the slides presenting this example. The slides with the animations are in TODO: ref to slides.

Double pendulum on cart



This example focuses on how to define the bodies in the system and the simulation



WPTO - NREL

Example initialization: do not modify

Do not modify:

- ❖ Defines time as symbolic variable and is used in the checks sections
- ❖ Define the dictionary for points and forces

```
# Do not modify
t = sym.symbols('t', real=True)
Initial_Points = {}
Force = {}
# End of Do not Modify
```



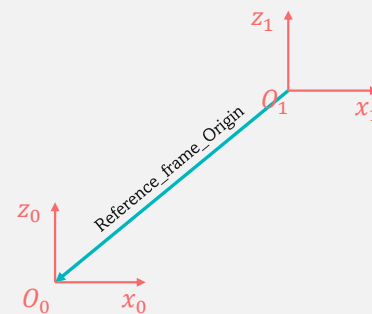
WPTO - NREL

Example initialization: user-defined

Gravity and origin:

- ❖ g is the gravity which can be set to any value, zero is applicable if floating in water
- ❖ `Reference_frame_Origin`: vector from absolute origin to origin of body 1
(important when first body is floating)

```
# Example 4
g = 9.81 # gravity
Reference_frame_Origin = np.array([1,-1])
Reference_frame_Origin = -Reference_frame_Origin
```



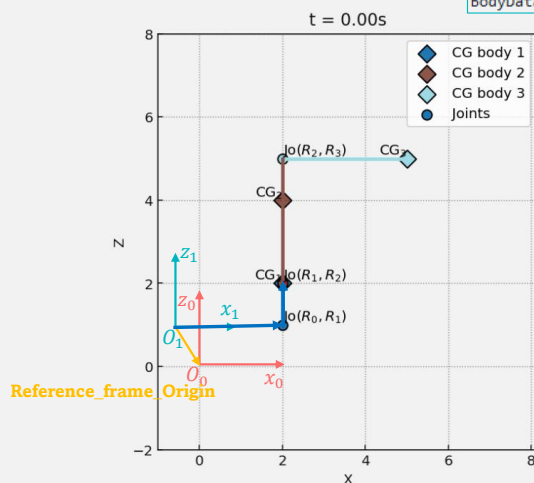
WPTO - NREL

3

Bodies definition: body 1

- Skip for now

```
# Bodies definition
dataNames = [] # Define symbolic variables for force points (for example, b1, b2, b3).
BodyDataSym = symVars_definition(dataNames, globals()) # DO NOT MODIFY
```



Connecting bodies

```
joints = [[0, 1], [1, 2], [2, 3]]
types = ['R', 'R', 'R']
parent_cg_to_joint = [[3, 0], [0, 0], [0, 1]]
joint_to_child_cg = [[0, 1], [0, 2], [3, 0]]
prismatic_direction = [[3, 0], [np.nan, np.nan], [np.nan, np.nan]]
```



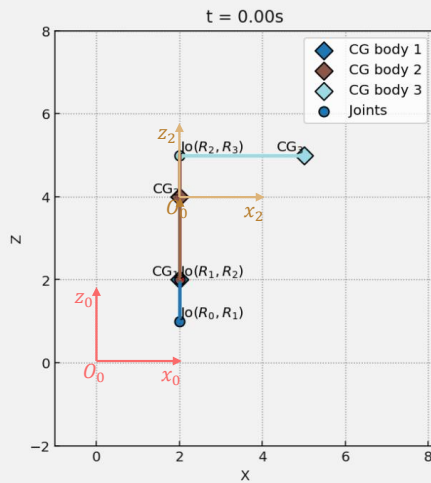
Go to next slide to continue animation



WPTO - NREL

4

Bodies definition: body 2



Connecting bodies

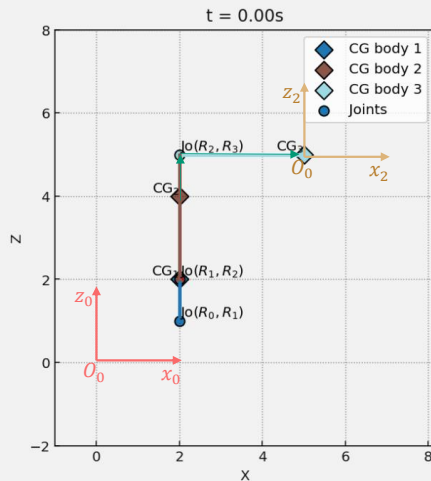
Joint type

```
joints = [[0, 1], [1, 2], [2, 3]]
types = ['P', 'R', 'R']
parent_cg_to_joint = [[3, 0], [0, 0], [0, 1]]
joint_to_child_cg = [[0, 1], [0, 2], [3, 0]]
prismatic_direction = [[3, 0], [np.nan, np.nan], [np.nan, np.nan]]
```

$S_1^{R_1, R_2}$
 $S_2^{R_1, R_2}$

Go to next slide to continue animation

Bodies definition: body 3



Connecting bodies

Joint type

```
joints = [[0, 1], [1, 2], [2, 3]]
types = ['P', 'R', 'R']
parent_cg_to_joint = [[3, 0], [0, 0], [0, 1]]
joint_to_child_cg = [[0, 1], [0, 2], [3, 0]]
prismatic_direction = [[3, 0], [np.nan, np.nan], [np.nan, np.nan]]
```

$S_2^{R_2, R_3}$
 $S_3^{R_2, R_3}$

Go to next slide to continue animation

Bodies definition: table display

Summary of how bodies were defined but organized by rows instead of columns

Bodies and joints table:

Joint	Connected	Bodies	Joint Type	ParentCGtoJoint	JointtoChildCG	Prismatic	Direction
1	0	1	P	3.000	0.000	0.000	1.000
2	1	2	R	0.000	0.000	0.000	2.000
3	2	3	R	0.000	1.000	3.000	0.000

1.000	0.000
NaN	NaN
NaN	NaN

Normalized direction

Simulation and Animation

Simulation

TimeStep = 0.05

tspan = 0.05

gVec = np.ones((len(types),1))

m0 = np.ones(len(types))

J0 = 2.*np.ones(len(types))

Factor to apply a different gravity to each body

Mass and Inertia of each body

Animation

animation_on

= 0

SaveMovieOn

= None

plotTstep

= 5

Boolean to display animation or not

Name with .gif extension if you want to save animation (requires animation_on = 1), None otherwise

Number of timeSteps to skip (integer) when displaying animation (Plots faster)

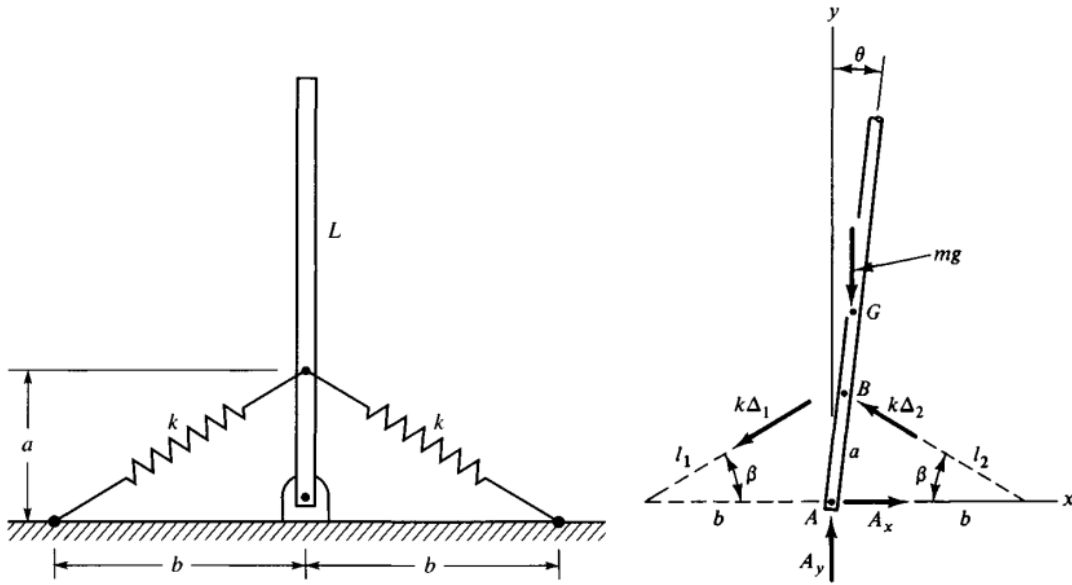
3.4 Validation of EOM derivation

3.4.1 Example 1: Oyster device

This example is done all symbolically and verified against one of the examples in Ginsberg's Advanced Dynamics Text book. The example introduces some simplifications that the code doesn't, therefore for valid comparison some numerical values must be substituted. The main assumption is small angle variation around equilibrium position.

The problem statement is:

Example 7.13 The bar is in static equilibrium in the upright position shown. The unstretched length of the identical springs is $l_0 < (a^2 + b^2)^{1/2}$, in order that the springs remain taut throughout any small displacement away from this position. Derive the corresponding equation of motion.



Example 7.13

Free-body diagram.

The derivation of the kinetic and potential energy is defined below for small θ and assuming $\cos^2 \beta = \frac{b^2}{(a^2 + b^2)}$

Solution We select as the generalized coordinate the angle θ relative to the vertical. Because $\theta = 0$ at the static equilibrium position, this variable serves directly as a relative generalized coordinate. The bar is in pure rotation about end A , so the kinetic energy is

$$T = \frac{1}{2}(\frac{1}{3}mL^2)\dot{\theta}^2.$$

This expression needs no simplification, because it is quadratic in $\dot{\theta}$.

We select the elevation of the pin as the datum for the gravitational potential energy. The springs also are conservative, so

$$V = \frac{1}{2}k\Delta_1^2 + \frac{1}{2}k\Delta_2^2 + \frac{1}{2}mgL \cos \theta.$$

In order to obtain some general properties regarding the potential energy of a spring, we shall consider the springs individually. We use the law of cosines to describe the length of each spring at an arbitrary position in terms of the generalized coordinate:

$$l_1 = [a^2 + b^2 - 2ab \cos(90^\circ + \theta)]^{1/2} = [a^2 + b^2 + 2ab \sin \theta]^{1/2},$$

$$l_2 = [a^2 + b^2 - 2ab \cos(90^\circ - \theta)]^{1/2} = [a^2 + b^2 - 2ab \sin \theta]^{1/2}.$$

We need terms only up to quadratic in the elongation, so we drop cubic and higher-degree terms in the expansion of the sine, as well as in the binomial expansion of the square root. The resulting elongation is

$$\begin{aligned} \Delta_1 &= l_1 - l_0 \\ &\approx [(a^2 + b^2)^{1/2} - l_0] + ab\theta(a^2 + b^2)^{-1/2} - \frac{1}{2}(ab\theta)^2(a^2 + b^2)^{-3/2}, \\ \Delta_2 &\approx [(a^2 + b^2)^{1/2} - l_0] - ab\theta(a^2 + b^2)^{-1/2} - \frac{1}{2}(ab\theta)^2(a^2 + b^2)^{-3/2}. \end{aligned}$$

Some terms cancel when we form Δ_1^2 and Δ_2^2 , with the result that the corresponding potential energy of each spring is

$$\begin{aligned} V_1 &= V_1^* + kab\theta \left[1 - \frac{l_0}{(a^2 + b^2)^{1/2}} \right] + \frac{1}{2} \frac{ka^2b^2l_0}{(a^2 + b^2)^{3/2}} \theta^2, \\ V_2 &= V_2^* - kab\theta \left[1 - \frac{l_0}{(a^2 + b^2)^{1/2}} \right] + \frac{1}{2} \frac{ka^2b^2l_0}{(a^2 + b^2)^{3/2}} \theta^2, \end{aligned}$$

where we have dropped the higher-order terms. When we add the potential energy in each spring, we find that the terms that are linear in θ cancel. This is a consequence of the fact that the linear terms characterize the static forces affecting the equilibrium position, $\theta = 0$, because only they give rise to a nonzero value of $(\partial V / \partial \theta)^*$. The static forces in each spring cancel. We may also consider V_1^* and V_2^* to be zero. An interesting form of the total potential energy of the springs results from expressing the horizontal distance b in terms of the angle of elevation β , from which we find that

$$V_1 + V_2 = \frac{kl_0(a \cos \beta)^2}{(a^2 + b^2)^{1/2}} \theta^2.$$

In the small-displacement approximation, $a\theta$ is the horizontal movement of point B . Then $a\theta \cos \beta$ is the component of this displacement parallel to either spring, which is the same as the linearized change in the length of either spring relative to its length at the equilibrium position. This observation is illustrative of the following general property.

Lastly, the equation of motion is defined by:

- ◆ *Suppose a spring is stretched very little at the static equilibrium position ($l_0 \approx (a^2 + b^2)^{1/2}$ in the present case). Then the quadratic terms in the potential energy of that spring may be obtained with reasonable accuracy by constructing linearized displacement components parallel to the spring at both of its ends.† Note that, if the stiffness k is large, this condition can be obtained even if the spring force at the equilibrium position is large.*

We now return to the derivation of the equation of motion. For the gravitational potential energy, we expand $\cos \theta$ in a series for small θ , and truncate that series at the second degree, which yields $\cos \theta \approx 1 - \theta^2/2$. The total potential energy that results is

$$V = \left[\frac{kl_0(a \cos \beta)^2}{(a^2 + b^2)^{1/2}} - \frac{1}{4}mgL \right] \theta^2 + \text{higher-order terms}.$$

It is interesting to note that the appearance of l_0 in this expression is surprising for some who have learned linearization concepts as ad hoc procedures.

The only forces remaining are the reactions, whose constraints are not violated. Therefore, $Q_1 = 0$. Substitution of T and V into Lagrange's equations therefore leads to

$$\frac{1}{3}mL^2\ddot{\theta} + \left[\frac{2kl_0(a \cos \beta)^2}{(a^2 + b^2)^{1/2}} - \frac{1}{2}mgL \right] \theta = 0.$$

This example is defined in the Example folder under the name “Flap_sandia”. To run it, update the main file to:

```
from Examples import Flap_sandia as ex
```

After running the example, we can compare the EOM given by the code:

```
sym.simplify(MBDsys.ReducedM)
```

Where we get the expression:

$$ReducedM = (m_1 * L^2)/4 + J_1$$

Recall that the moment of inertia, J_1 , is always around the CG and for a bar is $J_1 = m_1 L^2/12$. So adding up both terms, the result matches the one from the book.

Right-hand-side:

The first step is to declare all necessary variables as symbolic:

```

from sympy import symbols, cos, diff

# define a symbol q
Theta_1, a, b, k, l0, m1, beta, g, L = symbols('Theta_1 a b k l0 m1 beta g L
→', real=True)

```

The expression from the book, for the right-hand-side without including θ , i.e. a stiffness, K , can be written as:

```

book_eq = 2 * k * l0 * (a * cos(beta))**2 / (a**2 + b**2)**0.5 - 0.5 * m1 *
g * L

```

The equivalent stiffness from the code can be compute as:

```

K = -sym.diff(MBDsys.Right_side,MBDsys.Q[0])

```

Where the minus comes from being on the other side of the equal sign. Since comparing these two equations in analytic form is not an easy task (left to the adventurous reader) we provide some numerical values for comparison.

$$a = 0.5, b = 1, k = 1, L = 2, m_1 = 1, l_0 = \sqrt{(a^2 + b^2)}, \beta = \cos\left(\frac{b^2}{a^2 + b^2}\right)^{-1}$$

Double check cos the answer matched at -9.41

```

book_eq.subs({L: 2, m1: 1, k: 1, a: 0.5, b: 1, l0: np.sqrt(1.25), beta:
betaVal, g:9.81}).evalf(4) = -9.49

```

The solution in the book is valid for small values of θ , as a result we substitute it by $\theta = 0.001$

```

K.subs({L: 2, m1: 1, k: 1, a: 0.5, b: 1, l0: np.sqrt(1.25), beta: betaVal,
MBDsys.Q[0]:1e-3}).evalf(4) = -9.41

```

Since both the right-hand-side and left-hand-side match we can conclude that the code is properly writing the equations of motion for this example.

Step-by-step PPT

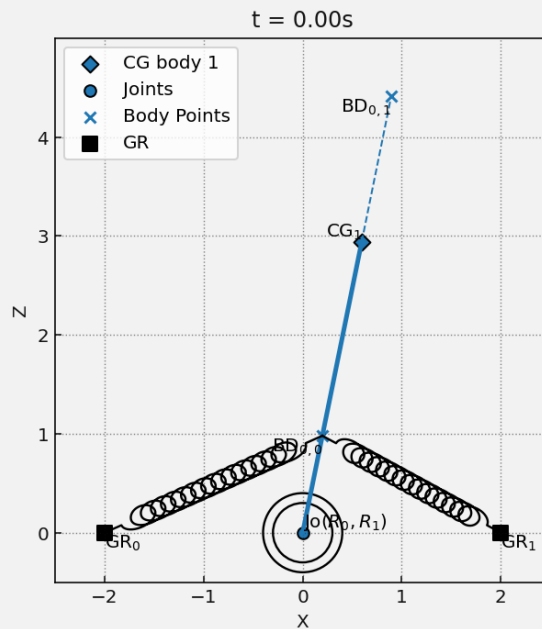
This portion contains the slides presenting this example. The slides with the animations are in TODO: ref to slides.

The following slides include also a torsion spring at the hinge. To fully reproduce the same example, leave the field empty.

```
Force['TorsionSpring'] = []
```

FLAP symbolic

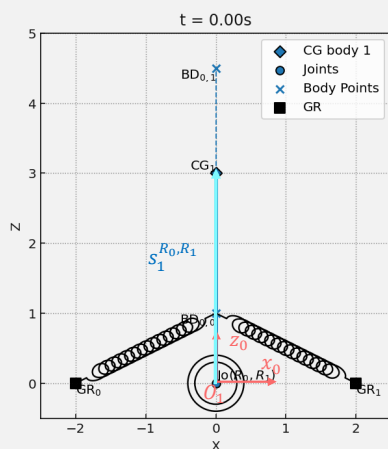
(previous example with symbolic parameters)



Bodies definition: symbolic flap's length, L

Define a list of strings ['L'] which are transformed into variables (sympy symbol)

```
dataNames = ['L']
BodyDataSym = symVars_definition(dataNames, globals())
```



Connecting bodies

```
joints = [[0, 1]]
types = [[R]]
parent_cg_to_joint = [[0, 0]]
joint_to_child_cg = [[0, L/2]]
prismatic_direction = [[hp, nan]]
```

Joint type

Direction of allowed translation

S_1

R_0, R_1

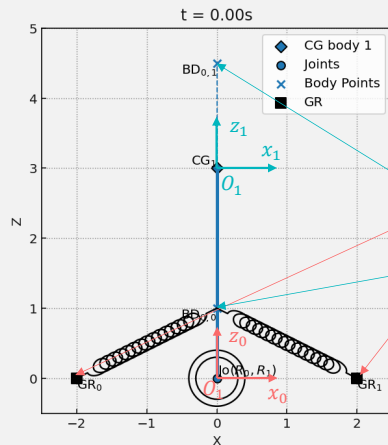
R_1, R_2

Go to next slide to continue animation

Points definition:

Define a list of strings ['a', 'b'], which are transformed into variables (sympy symbol)

```
PointNames = ['a', 'b']
PointsSym = symVars_definition(PointNames, globals())
```



Note: vectors measured from the appropriate origin (see colors)

```
Initial_Points["GR"] = [[-b, 0], [b, 0]]
```

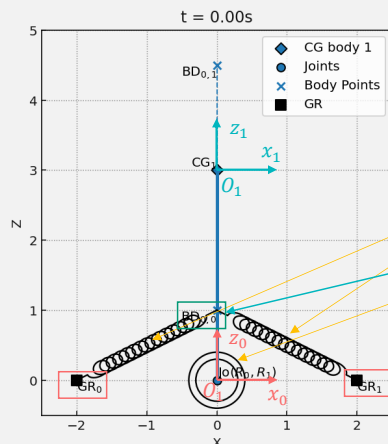
```
Initial_Points["BD"] = {}
Initial_Points["BD"][1] = [[0, a-L/2], [0, L/2]]
```

If not specified, the symbolic variables are given the value of 1 on evaluation.

Go to next slide to continue animation

Forces definition: symbolic stiffness, k

Springs and dampers are defined as a list of tuples



```
Force["TensionSpring"] = [((["BD10", "GR00"], [2.2361, k]),
                             (["BD10", "GR01"], [2.2361, k]),
                             )]
```

```
Force["TorsionSpring"] = [((["0,1"], [0, 100])]
```

Undeformed length, l_0

Stiffness, k

Undeformed angle, θ_0

Stiffness, k_θ

Go to next slide to continue animation

DEVELOPER'S DOCUMENTATION

In case of contributing to this project, there are two files that must be run to check everything is working as it should. These files are under `multibody.Validation`. The first file, `Automated_validation`, checks that the position of the CGs for each body is correct as well as the joint location and the R and $\dot{R}$ matrices for Examples 1-8. The comparison is done to the True values stored in `.json` files under `TrueMBDvars` folder in the same location. The second set of checks, compares the energy of the system to a set of true values saved in `TrueEnergy`. In this case the variables are saved as `.mat` files. This file is called `EnergyandODE_comparison` and it should provide values around or below 10^{-5}

Both these files can be run directly from this location.

In summary, from the root do the following:

```
cd source/multibody/Validation
python Automated_validation.py
python EnergyandODE_comparison.py
```

Whether you use forward or backward slash will depend on the terminal and OS you use.

4.1 Project Layout

The following shows the structure of the `multibody` package:

```
multibody/
  Checks/
    Body_check.py
    Points_forces_check.py
```

(continues on next page)

(continued from previous page)

```

ExtForcesManager/
    manager.py
    TODO: move MoorDyn here
MultibodyCore/
    branch_calc_graph_based.py
    Joints_helpers_class.py
    joints_system.py
    kinematicsTransformation.py
    mbd_system.py
    PointsAndForce_definition.py
    RigidBodyIntegrator.py
    SpringsAndDampers_helpers_class.py
    SystemsEnergy.py
Plotting/
    plotting_all.py
    plotting_utilities.py
Symvars/
    symVars_py.py
Tables/
    PointsForces2table.py
Validation/
    Automated_validation.py
    EnergyandODE_comparison.py

```

4.2 Public API summary

The following symbols are exposed by default when importing the `multibody` package:

```

from multibody import (
    # High-level interface
    MBDSysytem,           # MultiBody dynamics class
                        # with all symbolic information
    MBdynamics,          # Integration in time
    ExtForcesManager,    # Enables coupling external libraries
                        # providing forces to the MBD code
)

```

(continues on next page)

(continued from previous page)

```

# Core classes & routines
JointSystem,          # Creates a class containing all bodies
                      # information, connectivity and variables
normalize_prismatic,  # Ensures the vectors of prismatic joints
                      # are unit length
VelTransformation,   # Computes Velocity transformation matrix and
its
                      # derivative, CG positions and joint positions
points_force_finder, # Assembles the generalized force vector
                      # and points-forces mapping
systems_energy,      # Computes system energy, assembles mass matrix
symVars_definition,  # Creates the symbolic variables defined in the
example

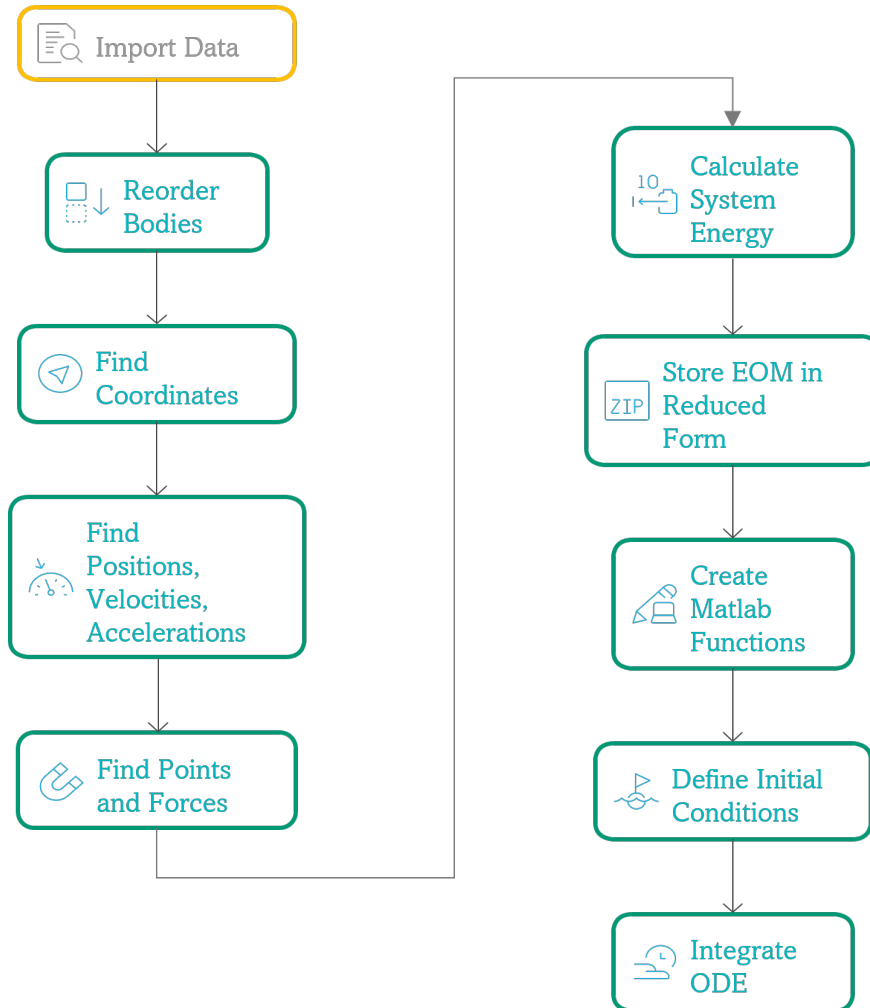
# Submodules (namespaces)
plot,                # Quick access to plotting tools
check,               # Quick access to checking tools
tables,              # Quick access to table-display tools
)

```

4.3 Multibody system creation workflow

The following workflow outlines the main steps followed by the multibody code. This is not necessary

Multibody Dynamics Simulation Workflow



4.4 PyTests

In case of contributing to this project, there are two files that must be run to check everything is working as it should. These files are under `multibody.Validation`. The first file, `Automated_validation`, checks that the position of the CGs for each body is correct as well as the joint location and the R and $\dot{R}$ matrices for Examples 1-8. The comparison is done to the True values stored in `.json` files under `TrueMBDvars` folder in the same location. The second set of checks, compares the energy of the system to a set of true values saved in `TrueEnergy`. In this case the variables are saved as `.mat` files. This file is called `EnergyandODE_comparison` and it should provide values around or below 10^{-5}

Both these files can be run directly from this location.

In summary, from the root do the following:

```
cd source/multibody/Validation
python Automated_validation.py
python EnergyandODE_comparison.py
```

Whether you use forward or backward slash will depend on the terminal and OS you use.

4.5 Public API

4.5.1 MultibodyCore package

This package contains the core multibody-dynamics engine: system setup, kinematics and symbolic equation generation. It also creates lambdified expressions for performance. The core element of this package is the `MBDsystem` class. The class is initialized:

- `from_example [method]`: reads an example where all the joints and connectivity is defined

<i><code>multibody.MBDSystem</code></i>	High-level multibody system combining symbolic model generation and numerical simulation.
<i><code>multibody.JointSystem</code></i>	Ordered collection of <code>Joint</code> objects plus convenience utilities for symbolic multibody modelling.
<i><code>multibody.normalize_prismatic</code></i>	Normalize a list of 2D prismatic-axis vectors into unit-length Sympy Matrices.
<i><code>multibody.VelTransformation</code></i>	Compute kinematic position and velocity transformations for a planar multibody system.
<i><code>multibody.points_force_finder</code></i>	Reads the points and forces defined in the dictionaries <code>Initial_Points</code> and <code>Forces</code> and parses them into the <code>MBDsystem</code> class.
<i><code>multibody.systems_energy</code></i>	Computes the total mechanical energy of the multibody system.
<i><code>multibody.symVars_definition</code></i>	Converts each string in the list to <code>sympy.Symbol</code> and saves it in the global workspace

MBDSystem

```
class MBDSystem(joints, types, parent_cg_to_joint, joint_to_child_cg,  
                prismatic_direction, Initial_Points, Force, Reference_frame_Origin,  
                ic, ForcesPointsSym, BodyDataSym, g=9.81, gVec=None)
```

Bases: `object`

High-level multibody system combining symbolic model generation and numerical simulation.

Constructs the full symbolic model (kinematics, constraints, forces, energies), compiles these to numeric functions, and provides an integration routine.

Parameters

- **joints** (*list[tuple[int, int]]*) – Parent-child connectivity for each joint.
- **types** (*list[str]*) – Joint types ('R', 'P', 'F', etc.).
- **parent_cg_to_joint** (*list*) – Vectors from parent body CG to joint locations.
- **joint_to_child_cg** (*list*) – Vectors from joint to child body CGs.
- **prismatic_direction** (*list*) – Unit direction vectors for prismatic joints.
- **Initial_Points** (*dict*) – Definitions of grounded and body-specific points.
- **Force** (*dict*) – Force definitions (CG forces, point forces, springs, dampers).
- **Reference_frame_Origin** (*tuple/list*) – Global origin coordinates.
- **ForcesPointsSym** (*list[sym.Symbol]*) – Symbolic variables for force-point magnitudes.
- **BodyDataSym** (*list[sym.Symbol]*) – Symbolic variables for body data (mass, inertia).
- **ic** (*np.ndarray*) – Initial conditions array for generalized coordinates and speeds.

- `g` (*float/sym.Symbol*) – Gravitational acceleration constant (default 9.81).
- `gVec` (*np.ndarray*) – Per-body gravity scale factors (default ones).

```
joints: list[tuple[int, int]]

types: list[str]

parent_cg_to_joint: list

joint_to_child_cg: list

prismatic_direction: list

Initial_Points: dict

Force: dict

Reference_frame_Origin: list | tuple

ic: ndarray

ForcesPointsSym: list[Symbol]

BodyDataSym: list[Symbol]

g: float | Symbol = 9.81

gVec: ndarray = None

joint_system: JointSystem

Q: list

QD: list

QDD: list

NDOF: int

Body_col: list

Pos = None

JointLoc = None
```

```
R = None

RD = None

Vel = None

Acc = None

Points_All = None

Force_All = None

Energy = None

M = None

m = None

J = None

Pos_func = None

JointLoc_func = None

R_func = None

RD_func = None

Vel_func = None

Acc_func = None

Right_side_func = None

ReducedM_func = None

Energy_func = None

BDpoints_func: dict

CGpoints_func = None

GRpoints_func = None

BDvel_func: dict
```

CGvel_func = None

t_update: list[int]

classmethod from_example(ex)

Create an MBDSysytem from a user-provided example module.

The example module must define: joints, types, parent_cg_to_joint, joint_to_child_cg, prismatic_direction, Initial_Points, Force, Reference_frame_Origin, ForcesPointsSym, BodyDataSym, ic, g (optional), and gVec (optional).

Return type

MBDSysytem

Parameters

ex (*ModuleType*)

integrate(mainNumVars, m_vals, J_vals, *, tspan, dt, rtol=1e-08, atol=1e-08, solver='RK45', external_manager=None)

Numerically integrate the equations of motion $\ddot{Q} = M^{-1} \cdot \text{RHS}$.

Parameters

- **mainNumVars** (*Sequence[float]*) – Flattened initial values for [Q, QD, force-point parameters, body data].
- **m_vals** (*Sequence[float]*) – Mass values per body.
- **J_vals** (*Sequence[float]*) – Inertia values per body.
- **tspan** (*float/tuple*) – End time or (start, end) for integration.
- **dt** (*float*) – Time step for output.
- **rtol** (*float*) – Solver tolerances.
- **atol** (*float*) – Solver tolerances.
- **solver** (*str*) – SciPy ODE solver name (e.g., “RK45”).
- **external_manager** – Optional force manager for external coupling.

Returns

Result object with time points t and solution y.

Return type
OdeResult

JointSystem

class JointSystem

Bases: `object`

Ordered collection of `Joint` objects plus convenience utilities for symbolic multibody modelling.

A `JointSystem` stores every joint in the mechanism, keeps them sorted by their child-body index, and exposes helper methods that bridge the gap between the tabular data a user typically writes and the richer objects required by the rest of the multibody package.

Notes

Ground is body 0. All other bodies are numbered consecutively starting at 1; joints are sorted so that the k-th entry in *joints* always corresponds to body k.

All vectors are 2-D column vectors expressed in the global X-Z plane (right-handed with Y pointing out of the screen).

joints

List of `Joint` instances, sorted by child index.

Type
`list[Joint]`

The following are generated by `:meth:`coordinate_finder``

Q, QD, QDD

System-wide symbolic coordinates, velocities, and accelerations.

Type
`list[sympy.Symbol]`

NDOF

Degrees of freedom contributed by each joint (1 for 'R'/'P', 3 for 'F').

Type
`list[int]`

Body_col

Map from joint $\rightarrow$ global DOF index(es). Useful when assembling matrices.

Type

list[list[int]]

Examples

```
>>> import sympy as sym
>>> import numpy as np
>>> from multibody import JointSystem, normalize_prismatic
>>>
>>> joints = [[0, 1], [1, 2]]
>>> types = ['F', 'R']
>>> pcg2j = [[0.0, 0.0], [1.5, 0.0]]
>>> j2ccg = [[np.nan, np.nan], [-1.0, 0.0]]
>>> pris_d = normalize_prismatic([[np.nan, np.nan],
...                               [np.nan, np.nan]])
>>>
>>> sys = JointSystem.from_data(joints, types, pcg2j, j2ccg, pris_d)
>>> sys.display_table()
>>> Q, QD, QDD, NDOF, Body_col = sys.coordinate_finder()
```

add_joint(parent, child, joint_type, parent_cg_to_joint, joint_to_child_cg, prismatic_direction=None)

Create a Joint instance and add it to the system.

classmethod from_data(joints, types, parent_cg_to_joint, joint_to_child_cg, prismatic_direction)

Build a JointSystem from arrays/lists corresponding to MATLAB inputs.

display_table()

Displays the joint system in a table format.

coordinate_finder()

Associates each joint with its corresponding symbolic coordinate variables for positions (Q), velocities (QD), and accelerations (QDD), based on the joint type. The joints are sorted by their child body number so that the DOF labeling index (used in the symbolic variable names) matches the child-body number.

Joint types:

- ‘R’ (Revolute): 1 DOF (rotational).
- ‘P’ (Prismatic): 1 DOF (translational).
- ‘F’ (Floating in 2D): 3 DOFs (X, Z, Theta).

Returns

List of symbolic variables for joint positions. QD (list): List of symbolic variables for joint velocities. QDD (list): List of symbolic variables for joint accelerations. DOF_list (list): List containing the DOF count for each joint. Body_col (list of lists): Each sublist contains the global DOF indices for the corresponding joint.

Return type

Q (list)

`multibody.normalize_prismatic`

normalize_prismatic(prismatic_direction)

Normalize a list of 2D prismatic-axis vectors into unit-length Sympy Matrices.

For each vector in prismatic_direction, this function:

1. Converts it to a sympy.Matrix.
2. Checks that it contains no nan entries and that its Euclidean norm is non-zero.
3. If valid, divides the vector by its norm to produce a unit-length axis.
4. Otherwise, returns Matrix([nan, nan]) for that entry.

Parameters

prismatic_direction (*Sequence[Sequence[float or sympy.Expr]]*) – Iterable of 2-element sequences specifying the local axis direction for each prismatic joint. Non-prismatic or uninitialized joints should be passed as [nan, nan].

Returns

A list of 2×1 sympy.Matrix objects, each of unit length (for valid inputs) or [nan, nan] if the original vector was invalid or zero-length.

Return type

List[sympy.Matrix]

VelTransformation

class VelTransformation(joint_system, Reference_frame_Origin)

Bases: object

Compute kinematic position and velocity transformations for a planar multibody system.

The VelTransformation class takes a populated *JointSystem* and a 2×1 reference-frame origin, then builds the global position vector **Pos**, the velocity-transformation matrix **R**, its time derivative **RD**, and a tracking matrix **R_track** for every body in the mechanism. It provides separate methods to handle joints grounded to the base (parent==0) and to propagate kinematics through non-grounded bodies.

Parameters

- **joint_system** (*JointSystem*) – Instance that supplies joint topology, symbolic coordinates, velocities, accelerations, DOF counts, and body-to-global index mappings.
- **Reference_frame_Origin** (*array-like or sympy.Matrix*) – 2×1 vector locating the global frame origin. If not already a *sympy.Matrix*, it will be converted internally.

Joints

List of [parent, child] index pairs for each joint.

Type

list[list[int]]

Type

Joint type codes ('R', 'P', 'F') in the same order as **Joints**.

Type

list[str]

ParentCGtoJoint

Vectors from each parent's center of gravity to its joint.

Type

list[sympy.Matrix]

JointtoChildCG

Vectors from each joint to its child's center of gravity.

Type
list[sympy.Matrix]

prismatic_direction

Unit-length axis vectors for prismatic joints, or [nan, nan] otherwise.

Type
list[sympy.Matrix]

NBodies

Number of moving bodies (ground is index 0, so maximum child index).

Type
int

Q, QD, QDD

Symbolic position, velocity, and acceleration variables extracted from joint_system.

Type
list[sympy.Symbol]

NDoF

Degrees of freedom contributed by each joint (1 for revolute/prismatic, 3 for floating).

Type
list[int]

Body_col

Maps each body (by index-1) to its global coordinate indices in Q, QD, QDD.

Type
list[list[int]]

Reference_frame_Origin

2×1 sympy vector representing the global frame origin after conversion.

Type
sympy.Matrix

Examples

```

>>> from sympy import nan
>>> from multibody.MultibodyCore.joints_system import JointSystem
>>> from multibody.MultibodyCore.kinematicsTransformation import
VelTransformation
>>> # assume `jsys` is a prebuilt JointSystem and `origin` a 2x1
vector
>>> vt = VelTransformation(jsys, origin)
>>> Pos, JointLoc, R, RD, R_track = vt.compute_grounded()
>>> Pos, JointLoc, R, RD, R_track = vt.compute_non_grounded(Pos,
JointLoc, R, RD, R_track)

```

compute_grounded()

Original method: returns (Pos, JointLoc, R, RD, R_track) for joints/bodies grounded to 0. This remains unchanged, so your call to ‘compute_grounded()’ still works.

compute_non_grounded(Pos, JointLoc, R, RD, R_track)

After calling compute_grounded() to get initial (Pos, JointLoc, R, RD, R_track), call this method to fill in the rest for bodies that have a non-zero parent.

Returns updated (Pos, JointLoc, R, RD, R_track).

multibody.points_force_finder

points_force_finder(Joints, NBodies, Pos, Q, QD, Initial_Points, Force, JointLoc, pris_body_to_parent_map, Body_col, Vel)

Reads the points and forces defined in the dictionaries Initial_Points and Forces and parses them into the MBDsystem class.

Parameters

- **data** (*list*) – Contains fields ‘joints_system.Joint’ (list of [parent, child] pairs) and ‘Type’ (list of joint types).
- **NBodies** (*int*) – Number of bodies in the system (1-based indexing for bodies).

- **Pos** (*sympy.Matrix*) – Global positions of all bodies stacked as [x1, z1, theta1, x2, z2, theta2, ...].
- **Q** (*list of sympy.Symbol*) – The symbolic position coordinates (e.g. [Theta_1, S_2, ...]).
- **QD** (*list of sympy.Symbol*) – The symbolic velocity coordinates (e.g. [ThetaD_1, SD_2, ...]).
- **Initial_Points** (*dict*) –
Must contain:
 - "GR": list of ground points (each [x, z]).
 - "BD": dict keyed by body number -> list of points [xRel, zRel], possibly symbolic.
- **Force** (*dict*) –
Dictionary of forces. May contain:
 - "PointsBD", "CG", "TensionSpring", "TensionDamper", "TorsionSpring", "TorsionDamper".
- **JointLoc** (*list of 2D coordinates*) – The location of each joint, consistent with data['Joints'] ordering.
- **pris_body_to_parent_map** (*list or dict*) – Info for prismatic joints' actual parent angles, as returned by the velocity routines.
- **Body_col** (*list of lists*) – For each body (1-based), the column indices in Q/QD that belong to it. E.g. Body_col[i-1] is a list of DOF indices for body i.
- **Vel** (*sympy.Matrix*) – Global velocities, parallel to Pos (stacked as [xD1, zD1, thetaD1, xD2, zD2, thetaD2, ...]).

Returns

- **Points_All** (*dict*) – Dictionary storing the computed positions for all relevant points (BD, GR, CG, JO).
- **Force_All** (*dict*) – Dictionary storing the computed force contributions for each body in each force category.

- `SpringPotentialEnergy` (`sympy.Expr`) – Total spring potential energy from tension/torsion springs.

`multibody.systems_energy`

`systems_energy`(NBodies, Q, QD, R, CGpos, g, gVec)

Computes the total mechanical energy of the multibody system. Correct only for linear springs and dampers for now.

Parameters

- **NBodies** (*int*) – Number of bodies in the system.
- **Q** (*list of sympy.Symbol*) – Generalized coordinates and velocities.
- **QD** (*list of sympy.Symbol*) – Generalized coordinates and velocities.
- **R** (*sympy.Matrix*) – Velocity transformation matrix.
- **CGpos** (*sympy.Matrix*) – Center-of-gravity positions for each body (rows: bodies, columns: x, z).
- **g** (*sympy.Symbol or float*) – Gravity constant.
- **gVec** (*list-like or sympy.Matrix*) – Per-body gravity-direction factors (e.g. 1 or 0).

Returns

- **Em** (`sympy.Expr`) – Total mechanical energy (kinetic + potential).
- **M** (`sympy.Matrix`) – Block-diagonal mass/inertia matrix.
- **m** (`sympy.Matrix`) – Mass vector for each body.
- **J** (`sympy.Matrix`) – Moment-of-inertia vector for each body.

`multibody.symVars_definition`

`symVars_definition`(varNames, scope_globals)

Converts each string in the list to `sympy.Symbol` and saves it in the global workspace

Parameters: - varNames: list of strings, each a variable name (e.g., ['x', 'y', 'z'])

Returns: - symbolicVars: sympy.Matrix column of symbolic variables - Also assigns each symbol into the global namespace (like assignin('base', ...))

4.5.2 Plotting package

This package is accessed through `multibody.plot` and it allows plotting figures using `plot_multibody_system` method or animations using `animate_multibody`. The method `pltTemplate` just has a set of predefined colors that work well with slides and have a light and dark backgrounds. None of these are fully black or fully white.

<code>multibody.plot.</code>	Draw the current configuration of a planar multibody system.
<code>plot_multibody_system</code>	Create and optionally save an animation of the multibody simulation.
<code>multibody.plot.animate_multibody</code>	Apply light or dark styling to a Matplotlib Axes to match MATLAB's PlotUtil.
<code>multibody.plot.pltTemplate</code>	

`multibody.plot.plot_multibody_system`

`plot.plot_multibody_system`(MBD, *, xfm=None, t_val=None, frame_bounds=None, dark_mode=False, ax=None, new_figure=True)

Draw the current configuration of a planar multibody system.

Renders bodies' centers of gravity, joints, body-defined points, ground points, forces (CG forces, point forces), springs, dampers, and optional mooring lines—all on a single static plot.

Parameters

- **main_num_vars** (*np.ndarray*) – Numeric array of values for every symbolic variable in `MBD.mainSymVars`. If `MBD.t_update` is not None and `t_val` is provided, the corresponding entry in this array is overwritten with `t_val`.
- **MBD** (*object*) – A multibody-dynamics container with attributes:
 - `mainSymVars`, `CGpoints_func`, `BDpoints_func`, `GRpoints_func`, `JOpoints_func`
 - `Force` (dict of force definitions)

- `t_update` (index of time variable or None)
- **`xfm`** (*optional*) – Transformation/adaptor for external libraries (e.g. MoorPy) used when plotting mooring lines.
- **`t_val`** (*float or None, optional*) – Current time in seconds. If provided, used to update the time symbol in `main_num_vars` and shown in the plot title.
- **`frame_bounds`** (*tuple of 4 floats or None, optional*) – Axis limits as (xmin, xmax, ymin, ymax). If None, limits are computed from the union of all plotted points with a 10% padding.
- **`dark_mode`** (*bool, optional*) – If True, applies dark-mode styling. Default is False.
- **`ax`** (*plt.Axes or None, optional*) – The Axes to draw on. If None or `new_figure` is True, a new Figure/Axes pair is created.
- **`new_figure`** (*bool, optional*) – If True, always create a fresh Figure/Axes even if `ax` is provided. Default is True.

Return type

`tuple[Figure, Axes]`

Returns

- `fig` (`plt.Figure`) – The Figure containing the multibody plot.
- `ax` (`plt.Axes`) – The Axes with all drawn elements.

`multibody.plot.animate_multibody`

`plot.animate_multibody`(`y`, `MBD`, `mainNumVars`, `frame_bounds=None`,
`save_path=None`, `fps=20`, `loop=False`, `xfm=None`)

Create and optionally save an animation of the multibody simulation.

Uses `plot_multibody_system` internally to render each frame.

Parameters

- **`tvec`** (*array-like, shape (n_frames,)*) – Time values for each frame.
- **`y`** (*array-like, shape (n_states, n_frames)*) – Simulation state array; rows are state variables, columns are time steps.

- **MBD** (*object*) – Multibody-dynamics container used by `plot_multibody_system`.
- **mainNumVars** (*array-like*) – Template array of numeric values for all symbolic variables. Updated each frame with `y` and (if needed) `t`.
- **frame_bounds** (*tuple of 4 floats, optional*) – Axis limits passed to the first call of `plot_multibody_system`.
- **save_path** (*str or Path, optional*) – If provided, path to write out the animation (via `PillowWriter`).
- **fps** (*int, optional*) – Frames per second for playback and saving. Default is 20.
- **loop** (*bool, optional*) – If True, the animation will loop when replayed. Default is False.
- **xfm** (*optional*) – Transformation/adaptor for external plotting (e.g. `MoorPy`).

Returns

`anim` – The `FuncAnimation` object, which can also be used to save or display the animation outside this function.

Return type

`matplotlib.animation.FuncAnimation`

`multibody.plot.pltTemplate`

`plot.pltTemplate(dark=False)`

Apply light or dark styling to a Matplotlib Axes to match MATLAB's `PlotUtil`.

Adjusts background color, tick parameters, title and axis label colors, grid appearance, and the default color cycle.

Parameters

- **ax** (*plt.Axes*) – The target Axes to style.
- **dark** (*bool, optional*) – If True, use dark-mode colors; otherwise use light-mode. Default is False.

Return type

`None`

4.5.3 Symvars package

<i><code>multibody.symVars_definition</code></i>	Converts each string in the list to <code>sympy.Symbol</code> and saves it in the global workspace
--	--

4.5.4 Checks package

This package checks if the example file is defined correctly. In particular, it verifies if the bodies/joints connectivity is valid, if the joint types are acceptable and if based on the joint type the position vectors are correct. Maybe add the wrong examples as documentation.

<i><code>multibody.check.bodies</code></i>	Validate a planar multibody joint table for structural and kinematic consistency.
<i><code>multibody.check.points_forces</code></i>	Validate point and force definitions for a planar multibody mechanism.

`multibody.check.bodies`

`check.bodies`(types, parent_cg_to_joint, joint_to_child_cg, prismatic_direction)

Validate a planar multibody joint table for structural and kinematic consistency.

This function mirrors MATLAB's `table_check`, performing the following checks:

1. Ground Connection
 - The first joint must connect to ground (parent index 0).
2. Table Completeness and Correctness
 - All input lists/arrays must have the same length.
 - Child body indices must form a continuous sequence (no gaps).
 - Each joint's parent must already appear as a body (no parentless children).
3. Ground-Related Checks
 - Every floating ('F') joint must attach directly to ground.

- For each joint type:
 - F: joint_to_child_cg must be [nan, nan].
 - R: parent_cg_to_joint must be fully defined (no nan).
 - P: joint_to_child_cg must be fully defined (no nan).

4. Prismatic-Direction Consistency

- For prismatic ('P') joints: direction vector must be normalized (unit length) and not contain nan.
- For non-prismatic joints: prismatic direction must be [nan, nan].

Parameters

- **joints** (*array-like of shape (n_joints, 2)*) – Parent-child index pairs for each joint (ground = 0).
- **types** (*sequence of str*) – Joint type codes for each joint: 'R' = revolute, 'P' = prismatic, 'F' = floating.
- **parent_cg_to_joint** (*sequence of array-like*) – Vectors from each parent body's center of gravity to its joint location.
- **joint_to_child_cg** (*sequence of array-like*) – Vectors from each joint to its child body's center of gravity.
- **prismatic_direction** (*sequence of array-like*) – Direction vectors for prismatic joints; other entries should be [nan, nan].

Returns

If all checks pass, prints a success message: "Table validation successful: No errors found."

Return type

None

Raises

ValueError – If any validation step fails, with a specific error message indicating:

- First joint is not grounded. - Inconsistent input lengths. - Missing or non-continuous body indices. - Parentless child definition. - Floating joint not grounded. - Incorrect nan usage in CG vectors for a given joint

type. - Prismatic direction not normalized or missing when required, or erroneously provided for non-prismatic joints.

`multibody.check.points_forces`

`check.points_forces`(types, Initial_Points, Force, t)

Validate point and force definitions for a planar multibody mechanism.

Performs these checks in order:

1. Adds missing keys in Force $\rightarrow$ ensures lists for 'PointsBD', 'CG', 'TensionSpring', 'TensionDamper', 'TorsionSpring', 'TorsionDamper'.
2. Adds missing keys in Initial_Points $\rightarrow$ ensures lists for 'BD', 'GR'.
3. Validates each entry in Force['PointsBD']:
 - Must be list-like with 2 elements.
 - Body and point indices are integers within valid ranges.
 - Referenced BD point exists and is not empty.
4. Validates each entry in Force['CG']:
 - Must be list-like with 2 elements.
 - Body indices are within [1, nBodies].
5. Validates TensionSpring and TensionDamper entries:
 - Entries come in (pointStr, param) pairs.
 - pointStr parsed via `check_point_string`.
 - param passes `check_parameter_has_no_function_of_t`.
 - Callable param is converted to symbolic form.
6. Validates TorsionSpring and TorsionDamper entries:
 - Same pairing and time-dependence rules.
 - Checks that any numeric/symbolic iterable has valid body indices.

Parameters

- **joints** (*array-like of shape (n_joints, 2)*) – List of [parent, child] index pairs.

- **types** (*sequence of str*) – Joint type codes ('R', 'P', 'F') matching joints.
- **Initial_Points** (*mapping or object*) –
Must supply:
 - Initial_Points['BD']: body-defined point lists
 - Initial_Points['GR']: ground reference point list
- **Force** (*dict-like*) –
Force definitions with keys:
'PointsBD', 'CG', 'TensionSpring', 'TensionDamper', 'TorsionSpring', 'TorsionDamper'
- **t** (*sympy.Symbol*) – The symbolic time variable for detecting unwanted time-dependence.

Returns

Prints success messages if all validations pass.

Return type

None

Raises

ValueError – If any of the above checks fail, with an explanatory message.

4.5.5 Tables package

The goal of this package is to allow the user to display on screen the points and forces defined on example for visualization. The bodies are automatically displayed from the JointSystem class. Maybe it would be good to move the bodies under Table module for consistency.

<code>multibody.tables.PointsTable</code>	Build a pandas DataFrame listing all defined points in the system.
<code>multibody.tables.ForceTable</code>	Build a consolidated pandas DataFrame of all force entries.

`multibody.tables.PointsTable`

`tables.PointsTable()`

Build a pandas DataFrame listing all defined points in the system.

Parameters

Initial_Points (*dict*) – Dictionary with two entries:

- "GR"
[list of array-like, shape (2,)] Ground points, each given as [x, z].
- "BD"
[dict of {int: list of array-like, shape (2,)}] Body-defined points.
Keys are body indices (>0), values are lists of points [x_rel, z_rel]
relative to that body's center of gravity.

Returns

df_points – Table with columns:

- Body
[int] Body index (0 for ground points, >0 for body-defined points).
- PointType
[str] Either "GR" (ground) or "BD" (body-defined).
- PointID
[int] Index of the point within its category list.
- Rel coordinates
[array-like, shape (2,)] The [x, z] coordinates (absolute for ground,
relative for body points).

Return type

pandas.DataFrame

`multibody.tables.ForceTable`

`tables.ForceTable()`

Build a consolidated pandas DataFrame of all force entries.

Parameters

Force (*dict of lists*) – Dictionary mapping force categories to their definitions:

- **PointsBD**
[list of lists] Each entry is [body, ptID, Fx, Fy, Mz].
- **CG**
[list of lists] Each entry is [body, Fx, Fy, Mz].
- **TensionSpring**
[list of tuples] Each entry is ((ptStr1, ptStr2), [l0, stiffness]).
- **TensionDamper**
[list of tuples] Each entry is ((ptStr1, ptStr2), damping_value).
- **TorsionSpring**
[list of tuples] Each entry is ([body1, body2], [theta0, stiffness]).
- **TorsionDamper**
[list of tuples] Each entry is ([body1, body2], damping_value).

Returns

df_forces – Table with columns:

- **ForceType**
[str] One of “PointsBD”, “CG”, “TensionSpring”, “TensionDamper”, “TorsionSpring”, “TorsionDamper”.
- **BodiesConnected**
[str] Comma-separated body IDs (e.g. “2” or “1,3”).
- **PointType**
[str] Point category code (e.g. “BD”, “CG”, “BD,BD”, “GR,BD”, “JO”).
- **PointID**
[str] Numeric point IDs (or “NA” if none).
- **ForceComponents**
[list or None] [Fx, Fy, Mz] for point-based forces, else None.
- **SpringParameters**
[list or None] [l0, stiffness] for springs, else None.
- **DamperParameters**
[scalar or None] Damping coefficient for dampers, else None.

Return type
pandas.DataFrame

4.5.6 ExtForcesManager subpackage

The ExtForcesManager handles coupling between the multibody core and external-force models (e.g. MoorDyn lines, wave loads, etc.). The multibody framework provides state variables such as position and velocity, whereas the external modules provide applied forces.

<i>multibody.manager. ExternalForcesManager</i>	Orchestrates several adapters that talk to third-party libraries (MoorDyn, FAST, CFD, ...).
---	---

ExternalForcesManager

class ExternalForcesManager(mbd_sys)

Bases: **object**

Orchestrates several adapters that talk to third-party libraries (MoorDyn, FAST, CFD, ...). Each adapter must implement:

`.initialize(t0, q0, qd0)` # optional `.update(t, q, qd)` # advance internal state
`.forces()` -> list[CartForce] # Cartesian forces in body frames

register(adapter)

Attach an adapter instance that follows the contract above.

Return type

None

get_adapter(name)

Return the adapter object previously registered under name. Raises KeyError if no such adapter exists.

Parameters

name (*str*)

initialize(t0, q0, qd0)

Call initialize method if needed for some reason (Optional)

Parameters

- `t0` (*float*)
- `q0` (*(`_SupportsArray[dtype[Any]]` / `_NestedSequence[_SupportsArray[dtype[Any]]]` / `bool` / `int` / `float` / `complex` / `str` / `bytes` / `_NestedSequence[bool / int / float / complex / str / bytes]`)*)
- `qd0` (*(`_SupportsArray[dtype[Any]]` / `_NestedSequence[_SupportsArray[dtype[Any]]]` / `bool` / `int` / `float` / `complex` / `str` / `bytes` / `_NestedSequence[bool / int / float / complex / str / bytes]`)*)

end()

Calls the `.end()` method on each adapter if it exists. (Optional)

init_plot()

Calls `.init_plot()` on each adapter if it exists. (Optional)

generalized_forces(`t`, `main_num_vars`)

Generate the generalized-coordinate force vector ΔRHS from external Cartesian forces.

This method:

1. Calls each registered adapter's `update(t, q, qd, main_num_vars)` to advance its internal state.
2. Retrieves Cartesian forces via `adapter.forces()`, which must return a list of `CartForce` namedtuples with fields:

- `pType` (*str*):
 - 'CG' for forces applied at a body's center of gravity (no moment generated).
 - 'BD' for forces applied at a body-defined point (moment computed about the y-axis).
- `body` (*int*): 1-based child body index in the joint system.
- `localID` (*int*): index into `MBD.BDpoints_func[body]` for 'BD'; ignored for 'CG'.

- `vector` (array_like of float, shape (2,)): planar force components `[Fx, Fz]` in the local body frame.
3. For each `CartForce`, computes a 3-element Cartesian force/moment vector `[Fx, Fz, moment]`, where `moment = 0` for `'CG'`, and `moment = -(rel_x*Fz - rel_z*Fx)` for `'BD'`.
 4. Assembles these into a global Cartesian vector `Fcart` of size `(3 * n_bodies, 1)`.
 5. Applies the numeric velocity-transformation matrix `R_func`: `Qgen = R_func(*main_num_vars).T @ Fcart`.

Parameters

- `t` (*float*) – Current time for which external forces should be evaluated.
- `main_num_vars` (*array_like of float*) – Concatenated numeric values of generalized coordinates (`Q`), velocities (`QD`), and any additional parameters required by the lambdified kinematic functions.

Returns

The generalized-force vector to add to the multibody RHS. If no external forces are provided, returns a zero vector.

Return type

`numpy.ndarray`, shape `(n_q,)`

Notes

- Adapters must be registered via `register(adapter)` and must implement:
 - `update(t, q, qd, main_num_vars)`
 - `forces()` -> `list[CartForce]`
- `CartForce` is defined as:

```
>>> CartForce = namedtuple("CartForce",
...                          ["pType", "body", "localID", "vector",
...                          "↪"])
```

- The `body` index is 1-based and corresponds to the child index in `JointSystem.joints`.
- For `pType=='BD'`, `localID` must reference a valid point from `MBD.BDpoints_func[body]`.

4.5.7 MoorDyn package

<code><i>multibody.MoorDyn.MBD2MoorDyn.MoorDynInterface</i></code>	Thin adapter that hides all MoorDyn details behind the 3-method contract expected by <code>ExternalForcesManager</code> .
<code><i>multibody.MoorDyn.MoorDyn_writer.write_moordyn_file</i></code>	

MoorDynInterface

```
class MoorDynInterface(mbd_sys, numericalValuesTuple, *, dt, MDline_types=None,
                       MDOptions=None, MDoutputs=None,
                       MDoutfile='berrinche.txt', MD_infile=None,
                       **writer_kwargs)
```

Bases: `object`

Thin adapter that hides all MoorDyn details behind the 3-method contract expected by `ExternalForcesManager`.

Parameters

- **mbd_sys** (*`multibody.MBDSystem` (to retrieve points/bodies)*)
- **force_dict** (the Force section from the user example)
- **MDline_types** (*(→ MoorDyn LINE TYPES section)*)
- **MDOptions** (*(→ MoorDyn OPTIONS)*)
- **MDoutfile** (path of the generated .txt)
- **writer_kwargs** (extra keywords forwarded to `write_moordyn_file`)
 - e.g. `bodies=...` if you want to override defaults
- **dt** (*float*)

```
• MDoutputs (list[str] / None)

• MD_infile (str / None)

name = 'moordyn'

update(t, q, qd, mainNumVars)

forces()

initialize(t0, q0, qd0)

end()

init_plot()
```

`multibody.MoorDyn.MoorDyn_writer.write_moordyn_file`

```
write_moordyn_file(out_path, *, line_types, points, lines, options=None,
                   outputs=None)
```

Parameters

- **out_path** (*str / Path*) – Destination .txt.
- **lines** (*Sequence[dict] / Mapping[int, dict]*) – Either list-of-dicts or dict keyed by ID → record-dict. Missing numeric fields default to zero.
- **options** (*Optional[Mapping[str, float | int | str]]*) – Mapping {name: value}. Preserves insertion order (Python 3.7).
- **outputs** (*Optional[Iterable[str]]*) – Iterable of output channel names (e.g. ["FairTen1", "FairTen2"]).
- **line_types** (*Sequence[dict] / Mapping[int, dict]*)
- **points** (*Sequence[dict] / Mapping[int, dict]*)
- **lines**

4.5.8 Validation package

The validation package compares the CG positions, Joint location, R-matrix and dot{R}-matrix for examples 1-8 to some true values. It is also possible to compare also the energy of the system to another set of true values, but this is correct only for systems with linear springs and dampers.

<i>multibody.Validation. Automated_validation</i>	Automated Validation of Multibody Exam- ples
<i>multibody.Validation. EnergyandODE_comparison</i>	Energy vs. ODE-solver Comparison

`multibody.Validation.Automated_validation`

Automated Validation of Multibody Examples

This script runs through a suite of predefined multibody system examples, computes key symbolic outputs via the public MBDSYSTEM class, and then compares them against “ground-truth” expressions stored as JSON.

For each Example 1–8, it:

1. Imports the example via `Examples.Example{i}`
2. Constructs the multibody system with `MBDSYSTEM.from_example(...)`
3. Loads the expected results (Pos, JointLoc, R, RD) from `TrueMBDvars/Example{i}.json`
4. Symbolically computes the difference, substitutes unit values, and sums absolute errors
5. Prints per-example error and a final summary list

Usage

Run on the command line (from the project root):

```
python Automated_validation.py
```

Or import and call individual functions (`load_expected`, `compare`) from another script.

Functions

```
compare(m_computed, m_expected)
```

```
is_not_nan(x)
```

```
load_expected(path)
```

```
load_expected(path)
```

```
is_not_nan(x)
```

```
compare(m_computed, m_expected)
```

```
multibody.Validation.EnergyandODE_comparison
```

Energy vs. ODE-solver Comparison

This script loads the “true” energy traces computed in MATLAB, runs the same examples through our Python `MBDSsystem.integrate(...)`, and then:

1. Resamples both time series onto a common grid
2. Computes max-abs and RMSE in total energy
3. Plots MATLAB vs. Python energy traces side by side

Usage

Just run:

```
python EnergyandODE_comparison.py
```

PYTHON MODULE INDEX

m

`multibody.Validation.Automated_validation,`
87

`multibody.Validation.EnergyandODE_comparison,`
88

A

Acc (MBDSystem attribute), 63
 Acc_func (MBDSystem attribute), 63
 add_joint() (JointSystem method), 66
 animate_multibody() (plot method), 74

B

BDpoints_func (MBDSystem attribute), 63
 BDvel_func (MBDSystem attribute), 63
 bodies() (check method), 76
 Body_col (JointSystem attribute), 65
 Body_col (MBDSystem attribute), 62
 Body_col (VelTransformation attribute), 69
 BodyDataSym (MBDSystem attribute), 62

C

CGpoints_func (MBDSystem attribute), 63
 CGvel_func (MBDSystem attribute), 63
 compare() (in module multi-body.Validation.Automated_validation), 88
 compute_grounded() (VelTransformation method), 70
 compute_non_grounded() (VelTransformation method), 70
 coordinate_finder() (JointSystem method), 66

D

display_table() (JointSystem method), 66

E

end() (ExternalForcesManager method), 83
 end() (MoorDynInterface method), 86
 Energy (MBDSystem attribute), 63
 Energy_func (MBDSystem attribute), 63
 ExternalForcesManager (class in multi-body.manager), 82

F

Force (MBDSystem attribute), 62
 Force_All (MBDSystem attribute), 63
 forces() (MoorDynInterface method), 86
 ForcesPointsSym (MBDSystem attribute), 62
 ForceTable() (tables method), 80
 from_data() (JointSystem class method), 66
 from_example() (MBDSystem class method), 64

G

g (MBDSystem attribute), 62
 generalized_forces() (ExternalForcesManager method), 83
 get_adapter() (ExternalForcesManager method), 82
 GRpoints_func (MBDSystem attribute), 63
 gVec (MBDSystem attribute), 62

I

- `ic` (MBDSystem attribute), 62
- `init_plot()` (ExternalForcesManager method), 83
- `init_plot()` (MoorDynInterface method), 86
- `Initial_Points` (MBDSystem attribute), 62
- `initialize()` (ExternalForcesManager method), 82
- `initialize()` (MoorDynInterface method), 86
- `integrate()` (MBDSystem method), 64
- `is_not_nan()` (in module `multibody.Validation.Automated_validation`), 88

J

- `J` (MBDSystem attribute), 63
- `joint_system` (MBDSystem attribute), 62
- `joint_to_child_cg` (MBDSystem attribute), 62
- `JointLoc` (MBDSystem attribute), 62
- `JointLoc_func` (MBDSystem attribute), 63
- `joints` (JointSystem attribute), 65
- `joints` (MBDSystem attribute), 62
- `Joints` (VelTransformation attribute), 68
- `JointSystem` (class in `multibody`), 65
- `JointtoChildCG` (VelTransformation attribute), 69

L

- `load_expected()` (in module `multibody.Validation.Automated_validation`), 88

M

- `M` (MBDSystem attribute), 63
- `m` (MBDSystem attribute), 63

MBDSystem (class in `multibody`), 61

module

- `multibody.Validation.Automated_validation`, 87
- `multibody.Validation.EnergyandODE_comparison`, 88

MoorDynInterface (class in `multibody.MoorDyn.MBD2MoorDyn`), 85

`multibody.Validation.Automated_validation` module, 87

`multibody.Validation.EnergyandODE_comparison` module, 88

`name` (MoorDynInterface attribute), 86

`NBodies` (VelTransformation attribute), 69

`NDOF` (JointSystem attribute), 65

`NDOF` (MBDSystem attribute), 62

`NDoF` (VelTransformation attribute), 69

`normalize_prismatic()` (in module `multibody`), 67

P

- `parent_cg_to_joint` (MBDSystem attribute), 62
- `ParentCGtoJoint` (VelTransformation attribute), 68
- `plot_multibody_system()` (plot method), 73
- `pltTemplate()` (plot method), 75
- `Points_All` (MBDSystem attribute), 63
- `points_force_finder()` (in module `multibody`), 70
- `points_forces()` (check method), 78
- `PointsTable()` (tables method), 80
- `Pos` (MBDSystem attribute), 62
- `Pos_func` (MBDSystem attribute), 63

`prismatic_direction` (MBDSystem attribute), [62](#)

`prismatic_direction` (VelTransformation attribute), [69](#)

Q

`Q` (MBDSystem attribute), [62](#)

`QD` (MBDSystem attribute), [62](#)

`QDD` (MBDSystem attribute), [62](#)

R

`R` (MBDSystem attribute), [62](#)

`R_func` (MBDSystem attribute), [63](#)

`RD` (MBDSystem attribute), [63](#)

`RD_func` (MBDSystem attribute), [63](#)

`ReducedM_func` (MBDSystem attribute), [63](#)

`Reference_frame_Origin` (MBDSystem attribute), [62](#)

`Reference_frame_Origin`
(VelTransformation attribute),
[69](#)

`register()` (ExternalForcesManager
method), [82](#)

`Right_side_func` (MBDSystem attribute),
[63](#)

S

`symVars_definition()` (in module multi-
body), [72](#)

`systems_energy()` (in module multibody),
[72](#)

T

`t_update` (MBDSystem attribute), [64](#)

`Type` (VelTransformation attribute), [68](#)

`types` (MBDSystem attribute), [62](#)

U

`update()` (MoorDynInterface method), [86](#)

V

`Vel` (MBDSystem attribute), [63](#)

`Vel_func` (MBDSystem attribute), [63](#)

`VelTransformation` (class in multibody), [68](#)

W

`write_moordyn_file()` (in module multi-
body.MoorDyn.MoorDyn_writer),
[86](#)